

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250285305

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Zhong; Yiran et al.

DEPTH ESTIMATION FOR AUTONOMOUS AND SEMI-AUTONOMOUS SYSTEMS AND APPLICATIONS

Abstract

In various examples, methods and systems are provided for estimating depth values for images (e.g., from a monocular sequence). Disclosed approaches may define a search space of potential pixel matches between two images using one or more depth hypothesis planes based at least on a camera pose associated with one or more cameras used to generate the images. A machine learning model(s) may use this search space to predict likelihoods of correspondence between one or more pixels in the images. The predicted likelihoods may be used to compute depth values for one or more of the images. The predicted depth values may be transmitted and used by a machine to perform one or more operations.

Inventors: Zhong; Yiran (Kirkland, WA), Loop; Charles (Mercer Island, WA), Smolyanskiy; Nikolai (Seattle, WA), Chen; Ke (Cupertino, CA), Birchfield; Stan (Sammamish, WA), Popov; Alexander (Kirkland, WA)

Applicant: NVIDIA Corporation (Santa Clara, CA)

Family ID: 87850816

Appl. No.: 19/213186

Filed: May 20, 2025

Related U.S. Application Data

parent US continuation 17592096 20220203 parent-grant-document US 12322126 child US 19213186

Publication Classification

Int. Cl.: G06T7/55 (20170101); G06T7/70 (20170101); G06V10/46 (20220101)

U.S. Cl.:

CPC **G06T7/55** (20170101); **G06T7/70** (20170101); **G06V10/462** (20220101);
G06T2207/20076 (20130101); G06T2207/20081 (20130101); G06T2207/30252
(20130101)

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of U.S. patent application Ser. No. 17/592,096, filed Feb. 3, 2022, which is incorporated herein by reference in its entirety.

BACKGROUND

[0002] Structure from Motion (SfM) may refer to the process of reconstructing the three-dimensional (3D) geometry of a scene from overlapping two-dimensional (2D) images generated by one or more cameras. In order to reconstruct the 3D geometry of a scene from the 2D images, the depth for the scene may be estimated from the 2D images and the camera poses used to generate the images. An existing deep learning approach may estimate depths and poses by formulating the problem as pose and monocular depth regression. In this approach, a convolutional neural network (CNN) is used for estimating a camera pose as a normalized pose using first and second images, and another convolutional neural network is used to independently estimate a normalized depth for the scene using only the first image. In most instances however, the estimates generated using this approach are lacking because it is difficult to accurately estimate depth from a single 2D image without additional information.

SUMMARY

[0003] Embodiments of the present disclosure relate to predicting image warping(s) for structure from motion using neural networks. In embodiments, relative camera pose estimation may be achieved by using geometry-based techniques, which results in a more robust machine learning solution for estimating depth.

[0004] In contrast to conventional approaches, such as those described above, disclosed approaches use camera pose information to define a search space between pairs of potential pixel matches between at least two frames, and predict likelihoods of correspondences between the pairs of potential matches to estimate a relative depth map. The pixels between frames (e.g., frames from a monocular sequence) may be matched based at least on the predicted likelihoods and used to estimate an essential matrix used to recover a relative camera pose. The recovered camera pose may then be used to triangulate matching points to estimate the depth map. In at least one embodiment, disclosed approaches may predict relative camera poses thereby avoiding problems related to regressing precise absolute scaled camera poses. In at least one embodiment, the camera pose used to define the search space may use an optical flow estimation network to predict correspondences between the frames. A mask may be generated to filter the predicted correspondences based at least on scale-invariant features of one or more images. For example, scale-invariant feature transform (SIFT) may be used where the predicted correspondences remaining after the mask correspond to SIFT keypoint locations (with the other predicted correspondences being masked out).

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0005] The present systems and methods for predicting image warping(s) for structure using a

neural network from motion are described in detail below with reference to the attached drawing figures, wherein:

[0006] FIG. 1A is a data flow diagram illustrating an example system performing a process for determining a depth map, in accordance with some embodiments of the present disclosure;

[0007] FIG. 1B is a data flow diagram illustrating an example system performing a process for determining a camera pose, in accordance with some embodiments of the present disclosure;

[0008] FIG. 2 is a data flow diagram illustrating an example system for performing a process for training a machine learning model for determining a depth map, in accordance with some embodiments of the present disclosure;

[0009] FIG. 3 depicts examples of ground truth depth data and predicted depth data for a process for determining a depth map, in accordance with some embodiments of the present disclosure;

[0010] FIG. 4 is a flow diagram showing a method for determining one or more depth values using a defined search space, in accordance with some embodiments of the present disclosure;

[0011] FIG. 5A is an illustration of an example autonomous vehicle in accordance with some embodiments of the present disclosure;

[0012] FIG. 5B is an example of camera locations and fields of view for the example autonomous vehicle of FIG. 5A in accordance with some embodiments of the present disclosure;

[0013] FIG. 5C is a block diagram of an example system architecture for the example autonomous vehicle of FIG. 5A, in accordance with some embodiments of the present disclosure;

[0014] FIG. 5D is a system diagram for communication between cloud-based server(s) and the example autonomous vehicle of FIG. 5A, in accordance with some embodiments of the present disclosure;

[0015] FIG. 6 is a block diagram of an example computing device suitable for use in implementing some embodiments of the present disclosure; and

[0016] FIG. 7 is a block diagram of an example data center suitable for use in implementing some embodiments of the present disclosure.

DETAILED DESCRIPTION

[0017] Systems and methods are disclosed related to predicting image warping(s) for structure from motion using a neural network. Although the present disclosure may be described with respect to an example autonomous vehicle **500** (alternatively referred to herein as “vehicle **500**” or “ego-machine **500**,” an example of which is described with respect to FIGS. 5A-5D), this is not intended to be limiting. For example, the systems and methods described herein may be used by, without limitation, non-autonomous vehicles, semi-autonomous vehicles (e.g., in one or more advanced driver assistance systems (ADAS)), piloted and un-piloted robots or robotic platforms, warehouse vehicles, off-road vehicles, vehicles coupled to one or more trailers, flying vessels, boats, shuttles, emergency response vehicles, motorcycles, electric or motorized bicycles, aircraft, construction vehicles, underwater craft, drones, and/or other vehicle types. In addition, although the present disclosure may be described with respect to automotive perception, this is not intended to be limiting, and the systems and methods described herein may be used in augmented reality, virtual reality, mixed reality, robotics, security and surveillance, autonomous or semi-autonomous machine applications, and/or any other technology spaces where depth information may be useful.

[0018] In disclosed approaches, any number of images or frames may be used to infer depth information, such as a first image comprising a first set of pixels and a second image comprising a second set of pixels generated using one or more cameras. In autonomous vehicle applications, the first and second images may be 2D images of road scenes generated using one or more cameras affixed to an autonomous vehicle. To estimate a camera pose (e.g., a relative camera pose) used to define the search space, the images may be applied to an optical flow estimator, such as a CNN or other machine learning model (MLM) trained to predict or infer correspondences between images using optical flow. For example, the MLM may perform feature extraction on each image in order to construct a feature map for each image and use the feature maps to predict likelihoods of

correspondence between points or pixels in the images (also referred to herein as predicted correspondences). Pairs of potential matching points or pixels may then be identified between the first image and the second image. In embodiments, dense matching points between each pixel in the first image and each pixel in second image may be generated using displacement-invariant cost learning (DICL).

[0019] To filter predicted correspondences used to compute the relative camera pose, a subset of pixels or points from the first image may be determined. In embodiments, the subset may comprise those pixels from the first image that have been determined to be invariant to at least one of scale or rotation. In one or more embodiments, the subset of pixels may correspond to keypoint locations generated using SIFT. The pairs of potential matching pixels corresponding to the subset of pixels may then be used to generate a relative camera pose(s). For example, a mask may be applied to the predicted correspondences to exclude those matching pixels which do not correspond to the subset of pixels in computing the camera pose. In embodiments, outlier matches may be filtered out prior to computing the camera pose(s). In examples, random sample consensus (RANSAC) may be used to identify and filter these outlier matches. The remaining matches may be used to retrieve the essential matrix and recover the relative camera pose.

[0020] Once the normalized camera pose has been estimated, the camera pose may be used to estimate the depth of the scene from the images (and/or other images). As described herein, in one or more embodiments, in order to estimate the depth from images, the images as well as the camera pose itself may be used in conjunction with an MLM(s)—such as a CNN-trained to predict likelihoods of correspondences between pairs of potential matches to estimate a relative depth map. For example, the camera pose may be used to define a search space comprising the pairs of potential matches between the images. In at least one embodiment, the camera pose(s) may be used to reduce the search space by leveraging geometric relations between 3D points and their projections onto the 2D frames. For example, the search space may be reduced to points along epipolar lines defined by projections of the cameras into each other's image planes. In non-limiting embodiments, the search space may have a maximum disparity of approximately 50 pixels, such that approximately 50 pixels in another image may be searched in trying to identify a match for each pixel in the image.

[0021] In embodiments, data representative of the pairs of potential matches from one or more of the images may be used by the MLM to predict—for each potential pair of matches—a likelihood that a corresponding pixel or point in one of the images corresponds to a corresponding pixel or point in another of the images. For example, the MLM may predict a cost volume where each cell may include a cost value(s) indicative of the likelihood for a pair of potential matches.

[0022] In examples, the pairs of potential matches may be in the form of a set of depth hypothesis planes that are encoded and input to the MLM. Using the camera pose, each pixel in an image may be projected onto (e.g., warped onto) potentially corresponding pixels in the search space for the other image(s) according to hypothetical depth planes (e.g., a stack of 2D arrays of values, where each 2D array contains the match likelihood for the pixels at a particular hypothesized depth, or, in other words, each 2D array of values captures different shiftings or warps of the pixels in one image to pixels or locations in another image according to a particular hypothesized depth).

[0023] In examples, given an image point or pixel x , L matching candidates

$$[00001] \{x'_l\}_{l=1}^L$$

may be generated at different hypothesized depths d_l according to Equation (1):

$$[00002] x'_l \sim K[R \cdot \text{Math. } \bar{t}][\frac{(K^{-1}x)d_l}{1}], \quad (1)$$

where K is the camera intrinsic matrix, R is the rotation matrix, t is a normalized translation vector t where $t \equiv t/\alpha$ such that $\|t\|_{\text{sub.2}}=1$, $d_{\text{sub.l}}=(L \times d_{\text{sub.min}})/l$, ($l=1, \dots, L$) is the depth hypothesis,

d.sub.min is a fixed minimum depth, and α is a scale factor used to determine the sampling distribution of matching candidates. By using the normalized translation vector t , the distribution of matching candidates may be invariant to scale, which is desirable because absolute scale is unknown. In further embodiments, a cost volume may be predicted by the MLM, which may include a matching cost for each warped pixel at all potential matching pixel locations within the search space. In examples, one or more cost aggregation methods may be used to refine each matching cost for a warped pixel at each pixel location using cost values for a 3×3 (or other sized) window of neighboring pixels surrounding each potential matching pixel.

[0024] In embodiments, one or more depth values may be computed using matched pixels in the images based at least on the likelihoods predicted for the pairs of potential matches. For example, a best match pair of the pairs of potential matches may be identified based at least on a lowest-matching cost value. The distance (e.g. the disparity) between the matched points or pixels may be used with the recovered camera pose (e.g., computed using the cost volume) to compute an estimated depth value.

[0025] In one or more embodiments, data may be transmitted to cause a machine (e.g., the autonomous vehicle) to perform one or more operations based at least on the one or more depth values. For example, data may be transmitted to cause an autonomous vehicle to navigate the environment while accounting for one or more objects or obstacles indicated by the estimated depth values. Although examples for estimating depth have been described in conjunction with defining the search space using a camera pose computed using optical flow estimation, any suitable technique may be used in conjunction with the present approaches for estimating depth by inferring image warpings.

[0026] Embodiments of the present disclosure also relate to systems and methods that leverage ground truth data to train and/or deploy MLMs to estimate a relative depth map for an autonomous machine. For example, data, such as LIDAR (light detection and ranging) data, representing various road scenes, may be converted into 2D images and used to train an MLM to more accurately estimate depth from 2D images.

[0027] In examples, to train the MLM, estimated depth values from the MLM may be scaled to match the ground truth depth values. To make the estimated and ground truth depths compatible, the estimated depth value d for a point or pixel may be matched to a ground truth depth d using Equation (2):

$$[00003] \quad d \sim \alpha_{\text{gt}} \hat{d}, \quad (2)$$

where α_{gt} refers to the ground truth scale.

[0028] Disclosed embodiments may be comprised in a variety of different systems (or platforms or applications) such as automotive systems (e.g., a control system for an autonomous or semi-autonomous machine, a perception system for an autonomous or semi-autonomous machine), systems implemented using a robot, aerial systems, medial systems, boating systems, smart area monitoring systems, systems for performing deep learning operations, systems for performing simulation operations, systems implemented using an edge device, systems incorporating one or more virtual machines (VMs), systems implemented at least partially in a data center, systems implemented at least partially using cloud computing resources, systems for performing light transport simulation, systems using assets at least partially developed with platforms for collaborative content creation using heterogeneous applications and/or for 3D assets, and/or other types of systems. While specific examples are provided, these example may be generalized beyond implementations details.

[0029] Now referring to FIG. 1A, FIG. 1A is a data flow diagram illustrating an example system performing a process **100** for determining a depth map, in accordance with some embodiments of the present disclosure. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces,

functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. In some embodiments, the systems, methods, and processes described herein may be executed using similar components, features, and/or functionality to those of example autonomous vehicle **500** of FIGS. **5A-5D**, example computing device **600** of FIG. **6**, and/or example data center **700** of FIG. **7**.

[0030] The system for determining a depth map may include a search space determiner **120**, an MLM(s) **122**, and a depth map determiner **126**. In examples, the depth map determiner **126** may also include an essential matrix **130A** used to recover the camera pose. In examples, the process **100** may include the system receiving input data **102**, which may include input data **104A** and input data **104B** (e.g., respectively representing one or more images), and outputting depth map data **132** (e.g., representing a depth map). In embodiments, the input data **104A** and the input data **104B** may represent 2D images. In one or more embodiments, the input data **104A** and/or the input data **104B** may represent consecutive 2D images taken from the vantage point of a moving vehicle. In one or more embodiments, the images may be from a monocular sequence, such as from a single camera. While one two images are shown any number of images may be used in various embodiments. Further one or more embodiments may use stereo images and/or one or more consecutive stereo images. The search space determiner **120** may determine a search space between pairs of potential pixel matches between the input data **104A** and the input data **104B**. In embodiments, the search space determiner **120** may use camera pose data **136** to determine or define the search space. In one or more embodiments, the camera pose data **136**, representing one or more camera poses, may be generated by a camera pose determiner **116** of FIG. **1B**. However, other approaches may be used to generate the camera pose data **136**.

[0031] Referring now to FIG. **1B** with FIG. **1A**, FIG. **1B** is a data flow diagram illustrating an example system performing a process **134** for determining a camera pose, in accordance with some embodiments of the present disclosure. The system for determining a camera pose may include an optical flow estimator **106**, a mask determiner **110**, a match masker **112**, and the camera pose determiner **116**. In examples, the camera pose determiner **116** may use an essential matrix **118** to determine the camera pose. In examples, the process **134** may include the system receiving the input data **102**, which includes the input data **104A** and the input data **104B**, and outputting the camera pose data **136**.

[0032] The optical flow estimator **106** may generate one or more disparity maps **108** between images based on the input data **102**, such as images represented by the input data **104A** and the input data **104B**. The optical flow estimator **106** may also predict likelihoods of correspondence between points or pixels represented by the input data **104A** and points or pixels represented by the input data **104B**.

[0033] In embodiments, the mask determiner **110** may create a mask that may be applied to the predicted correspondences between the images. In embodiments, the mask determiner **110** may determine a subset of the pixels or points that may be invariant to at least one of scale or rotation. The mask may be applied to the disparity map(s) **108** to exclude those matching pixels that do not correspond to the subset of pixels or points. In examples, the match masker **112** may apply the mask generated by the mask determiner **110** to identify matching pixels or points **114** between the input data **104A** and the input data **104B**. The camera pose determiner **116** may filter out outlier matches using techniques such as RANSAC to retrieve the essential matrix **118** from the masked disparity map(s) **108** and the camera pose data **136** may be determined using the essential matrix **118**.

[0034] Referring back to FIG. 1A, the camera pose(s) represented by the camera pose data **136** may be used by the search space determiner **120** to define the search space comprising pairs of potential matches between the input data **104A** and the input data **104B**. For example, the search space determiner **120** may use the camera pose to reduce the search space to points along epipolar lines defined by projections of cameras corresponding to the input data **104A** and the input data **104B** into each other's image planes. In non-limiting embodiments, the search space may have a maximum disparity of approximately 50 pixels, such that approximately 50 pixels in another image may be searched in trying to identify a match for each pixel in the image.

[0035] The search space determiner **120** may define the search space using a set of depth hypothesis planes **124**, which may be generated and input to the MLM(s) **122**. In addition, one or more portions of the input data **104A** and/or the input data **104B** and/or images represented by and/or derived therefrom may also be input to the machine learning model(s) **122**. Each pixel of the input data **104A** may be warped onto potentially corresponding pixels in the search space for the input data **104B** according to each of the depth hypothesis planes **124**. The MLM(s) **122** may predict a cost volume **128** for these warpings (e.g., with a cost value for each pair of pixels). The cost volume **128** may include a matching cost for each warped pixel at all potential matching pixel locations within the search space. The depth map determiner **126** may determine the depth map data **132** using matched pixels in the input data **104A** and the input data **104B**. In embodiments, a best match pair of the pairs of potential matches may be identified based on a lowest-matching cost value indicated by the cost volume **128** (or using different or additional selection criteria). The matched pixels may be used to generate the essential matrix **130A**, which may then be used to recover the relative camera pose. The depth map determiner **126** may then use the camera pose to triangulate matching points to estimate one or more depth maps represented by the depth map data **132**.

[0036] The one or more MLM(s) **122** may include any suitable type of MLM, such as an MLM(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, long/short term memory/LSTM, Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), lane detection algorithms, computer vision algorithms, and/or other types of MLMs.

[0037] As an example, such as where the MLM(s) **122** includes a convolution neural network (CNN), the CNN may include any number of layers. One or more of the layers may include an input layer. The input layer may hold values associated with data, such as input data **102** (e.g., before or after post-processing). For example, when the input data **102** represents an image, the input layer may hold values representative of the raw pixel values of the image(s) as a volume (e.g., a width, a height, and color channels (e.g., RGB), such as $32 \times 32 \times 3$).

[0038] One or more layers may include convolutional layers. The convolutional layers may compute the output of neurons that are connected to local regions in an input layer, each neuron computing a dot product between their weights and a small region they are connected to in the input volume. A result of the convolutional layers may be another volume, with one of the dimensions based on the number of filters applied (e.g., the width, the height, and the number of filters, such as $32 \times 32 \times 12$, if 12 were the number of filters).

[0039] One or more layers may include deconvolutional layers (or transposed convolutional layers). For example, a result of the deconvolutional layers may be another volume, with a higher dimensionality than the input dimensionality of data received at the deconvolutional layer.

[0040] One or more of the layers may include a rectified linear unit (ReLU) layer. The ReLU layer(s) may apply an elementwise activation function, such as the max (0, x), thresholding at zero, for example. The resulting volume of a ReLU layer may be the same as the volume of the input of the ReLU layer.

[0041] One or more of the layers may include a pooling layer. The pooling layer may perform a down sampling operation along the spatial dimensions (e.g., the height and the width), which may result in a smaller volume than the input of the pooling layer (e.g., $16 \times 16 \times 12$ from the $32 \times 32 \times 12$ input volume).

[0042] One or more of the layers may include one or more fully connected layer(s). Each neuron in the fully connected layer(s) may be connected to each of the neurons in the previous volume. The fully connected layer may compute class scores, and the resulting volume may be $1 \times 1 \times \text{number of classes}$. In some examples, the CNN may include a fully connected layer(s) such that the output of one or more of the layers of the CNN may be provided as input to a fully connected layer(s) of the CNN. In some examples, one or more convolutional streams may be implemented by the CNN(s), and some or all of the convolutional streams may include a respective fully connected layer(s).

[0043] In some non-limiting embodiments, the CNN(s) may include a series of convolutional and max pooling layers to facilitate image feature extraction, followed by multi-scale dilated convolutional and up-sampling layers to facilitate global context feature extraction.

[0044] Although input layers, convolutional layers, pooling layers, ReLU layers, and fully connected layers are discussed herein with respect to the CNN(s), this is not intended to be limiting. For example, additional or alternative layers may be used in the CNN(s), such as normalization layers, SoftMax layers, and/or other layer types.

[0045] In embodiments, different orders and numbers of the layers of the CNN may be used depending on the embodiment. In other words, the order and number of layers of the CNN(s) is not limited to any one architecture.

[0046] In addition, some of the layers may include parameters (e.g., weights and/or biases), such as the convolutional layers and the fully connected layers, while others may not, such as the ReLU layers and pooling layers. In some examples, the parameters may be learned by the CNN(s) during training. Further, some of the layers may include additional hyper-parameters (e.g., learning rate, stride, epochs, etc.), such as the convolutional layers, the fully connected layers, and the pooling layers, while other layers may not, such as the ReLU layers. The parameters and hyper-parameters are not to be limited and may differ depending on the embodiment.

[0047] Referring now to FIG. 2, FIG. 2 is a data flow diagram illustrating an example system for a process **200** for training an MLM using for determining a depth map, in accordance with some embodiments of the present disclosure. The system may include one or more of the components of the system described in connection with FIGS. 1A and/or 1B. For example, the system may include one or more of the search space determiner **120**, the MLM(s) **122**, and the depth map determiner **126**. In embodiments, the system may also include a training engine **202**. In examples, the process **200** may include the system: receiving training data **204** from the training engine **202**, which may include input data **206A** and input data **206B** (e.g., respectively representing one or more images); receiving from the training engine **202**, a ground truth camera pose (e.g., measured by the vehicle from which the training data **204** was generated from); and outputting depth map data **210**, which may then be used by the training engine **202** to update one or more parameters of the MLM(s) **122** based at least on ground truth data **212**.

[0048] In examples, the depth map data **210** may be generated using the inferences from the MLM **122** for the input data **206A** and the input data **206B**. In embodiments, the training data **204**, such as the input data **206A** and the input data **206B**, may be in the form of 2D images. The ground truth data **212** may represent one or more depth maps, such as a depth map image generated in association with the input data **206A** and the input data **206B** from LIDAR data. In examples, the ground truth data **212** may be derived from a moving vehicle with a known camera pose.

[0049] In embodiments, the search space determiner **120**, using the known camera pose supplied by the training engine **202**, may determine a search space defined by a set of depth hypothesis planes **206**, which may be input to the MLM(s) **122** (using approaches described with respect to FIG. 1A). In addition, one or more portions of the input data **204A** and/or the input data **204B** and/or images

represented by and/or derived therefrom may also be input to the machine learning model(s) **122**. Each pixel from the input data **206A** may be warped onto potentially corresponding pixels in the search space for the input data **206B** according to each of the hypothetical depth planes **206**. The MLM(s) **122** may predict a cost volume **208** for these warpings (e.g., with a cost value for each pair of pixels). The cost volume **208** may include a matching cost for each warped pixel at all potential matching pixel locations within the search space. The depth map determiner **126** may determine the depth map data **210** using matched pixels in the input data **206A** and the input data **206B**. In embodiments, a best match pair of the pairs of potential matches may be identified based on a lowest-matching cost value indicated by the cost volume **208** (or using different or additional selection criteria). The matched pixels may be used to generate the essential matrix **130B**, which may then be used to recover the relative camera pose. The depth map determiner **126** may then use the camera pose to triangulate matching points to estimate one or more depth maps represented by the depth map data **210**.

[0050] The depth map data **210** output from the system may be supplied to the training engine **202** to further train the MLM(s) **122**. For example, where the predicted depth values for the training data **204A**, such as the depth map data **210**, diverge from the ground truth depth values for the training data **204A**, such as the LIDAR depth values for the training data **204A**, the training engine **202** may supply the MLM(s) **122** with updated parameters to further train the MLM(s) **122** to predict depth. The training engine **202** may then compare the depth map data **210** to corresponding ground truth data **212** and update one or more parameters of the MLM **122** accordingly.

[0051] In one or more embodiments, the optical flow estimator **106** is trained jointly with the MLM **122**, or the optical flow estimator **106** may be trained separate from the MLM **122**. For example, the optical flow estimator **106** may be trained using the known or measured camera poses. In one or more examples, the optical flow estimator **106** may be trained using iterations of a similar process as the process **134**. When training the MLM **122**, while some embodiments may use known or measured camera poses, in other embodiments the search space determiner **120** may use camera poses inferred using the process **134**.

[0052] In at least one embodiment, where both a ground truth camera pose and ground truth depth information are given, the optical flow estimator **106** may be updated based at least on computing the rigid flow using the ground truth data. For example, the rigid flow may be used as a supervision signal in computing the loss function for the optical flow estimator **106**.

[0053] Referring now to FIG. 3 with FIG. 2, the top portion of FIG. 3 depicts an example of a 2D image **306** which may correspond to the input data **206A**, and which may be used to train the MLM(s) **122**. The bottom portion of FIG. 3 depicts an example of an image **306A** including a representation **308** of LIDAR data along with a representation **310** of at least some of the depth map data **210** corresponding to the input data **206A**. In one or more embodiments, the LIDAR data may correspond to the ground truth data **212** for the input data **206A**. For example, the training engine **202** may generate one or more ground truth depth maps from the LIDAR data and compare the one or more ground truth depth maps to one or more depth maps generated using the MLM(s) **122** accordingly in order to update one or more parameters thereof.

[0054] Now referring to FIG. 4, each block of method **400**, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. The method **400** may also be embodied as computer-usable instructions stored on computer storage media. The method **400** may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, method **400** is described, by way of example, with respect to FIGS. 1A-1B. However, these methods may additionally or alternatively be executed within any one process by any one system, or any combination of processes and systems, including, but not limited to, those described herein.

[0055] FIG. 4 is a flow diagram showing a method **400** for determining one or more depth values using a defined search space, in accordance with some embodiments of the present disclosure. The method **400**, at block **B402**, includes receiving a first image and a second image. For example, the search space determiner **120** may receive the first image and the second image. In embodiments, the first image and the second image may be consecutive 2D images of a road scene, such as those represented by the input data **102** and illustrated in FIG. 1A. In examples, the first image and the second image may be generated using a single camera. In embodiments, the images may be generated using one or more cameras attached to an autonomous vehicle.

[0056] The method **400**, at block **B404**, includes defining a search space of potential matches between the first and second images. In examples, the search space determiner **120** may define, based at least on a camera pose associated with the one or more cameras, a search space comprising sets of potential matches for first one or more pixels in the first image to second one or more pixels in the second image. The camera pose associated with the one or more cameras may be a relative camera pose estimated based at least on optical flow estimation operations applied to the first image and the second image. In an example, the camera pose may be generated using the process **134** illustrated in FIG. 1B. In another example, the camera pose may be generated using some other process. In one or more embodiments, the search space determiner **120** may define the search space based at least on determining one or more epipolar lines along the first one or more pixels in the first image based at least on the camera pose and selecting one or more of the pixels in the second image for inclusion in the sets of potential matches based at least on the one or more pixels corresponding to one or more locations along the one or more epipolar lines. In various examples, the search space determiner **120** may define the search space using scale-invariant translation vectors. In this way, the accuracy of a depth prediction may remain unchanged even if the overall scale of the scene represented in one image changes in a second image.

[0057] The method **400**, at block **B406**, includes using the potential matches and one or more of the images to predict a likelihood of correspondence between pixels in the first and second images. For example, first data representative of the sets of the potential matches and second data representative of the second image may be applied to the MLM **122** trained to predict for each set of the sets of potential matches, a likelihood that the first one or more pixels in the first image corresponds to at least one pixel of the second one or more pixels in the second image.

[0058] The method **400**, at block **B408**, includes computing one or more depth values using the predicted likelihood of correspondence. For example, the depth map determiner **126** may compute one or more depth values using locations that correspond to the first one or more pixels in the first image and the at least one pixel in the second image based at least on the likelihood predicted for the first one or more pixels in the first image and the at least one pixel in the second image.

[0059] The method **400**, at block **B410**, includes transmitting data to cause a machine to perform one or more operations based at least on the one or more depth values. For example, the transmitted data may cause a machine, e.g. an autonomous vehicle, to navigate the environment while accounting for one or more objects or obstacles indicated by the one or more depth values.

[0060] In embodiments, the first image and the second image may be applied to one or more MLM(s) trained to predict correspondences between the first image and the second image using an optical flow estimation. For example, the input data **104A** and the input data **104B** may be input to the optical flow estimator **106**. In other embodiments, a mask may also be determined based at least on analyzing scale-invariant features of one or more of the first image or the second image. For example, the mask determiner **110** may create a mask that comprises a subset of pixels corresponding to keypoint locations in the first image and/or second image generated using SIFT. In still other embodiment, the mask may be applied to determine a subset of the correspondences predicted using one or more MLMs. For example, the match masker **112** may filter out outlier matches of predicted correspondences using filtering techniques such as RANSAC, resulting in a subset of the predicted correspondences. In embodiments, the camera pose used for defining the

search space of potential matches between the first and second images may be computed using the subset of the predicted correspondences. For example, the camera pose determiner **116** may compute the camera pose using the output of the match masker **112**.

Example Autonomous Vehicle

[0061] FIG. 5A is an illustration of an example autonomous vehicle **500**, in accordance with some embodiments of the present disclosure. The autonomous vehicle **500** (alternatively referred to herein as the “vehicle **500**”) may include, without limitation, a passenger vehicle, such as a car, a truck, a bus, a first responder vehicle, a shuttle, an electric or motorized bicycle, a motorcycle, a fire truck, a police vehicle, an ambulance, a boat, a construction vehicle, an underwater craft, a drone, a vehicle coupled to a trailer, and/or another type of vehicle (e.g., that is unmanned and/or that accommodates one or more passengers). Autonomous vehicles are generally described in terms of automation levels, defined by the National Highway Traffic Safety Administration (NHTSA), a division of the US Department of Transportation, and the Society of Automotive Engineers (SAE) “Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles” (Standard No. J3016-201806, published on Jun. 15, 2018, Standard No. J3016-201609, published on Sep. 30, 2016, and previous and future versions of this standard). The vehicle **500** may be capable of functionality in accordance with one or more of Level 3-Level 5 of the autonomous driving levels. The vehicle **500** may be capable of functionality in accordance with one or more of Level 1-Level 5 of the autonomous driving levels. For example, the vehicle **500** may be capable of driver assistance (Level 1), partial automation (Level 2), conditional automation (Level 3), high automation (Level 4), and/or full automation (Level 5), depending on the embodiment. The term “autonomous,” as used herein, may include any and/or all types of autonomy for the vehicle **500** or other machine, such as being fully autonomous, being highly autonomous, being conditionally autonomous, being partially autonomous, providing assistive autonomy, being semi-autonomous, being primarily autonomous, or other designation.

[0062] The vehicle **500** may include components such as a chassis, a vehicle body, wheels (e.g., 2, 4, 6, 8, 18, etc.), tires, axles, and other components of a vehicle. The vehicle **500** may include a propulsion system **550**, such as an internal combustion engine, hybrid electric power plant, an all-electric engine, and/or another propulsion system type. The propulsion system **550** may be connected to a drive train of the vehicle **500**, which may include a transmission, to enable the propulsion of the vehicle **500**. The propulsion system **550** may be controlled in response to receiving signals from the throttle/accelerator **552**.

[0063] A steering system **554**, which may include a steering wheel, may be used to steer the vehicle **500** (e.g., along a desired path or route) when the propulsion system **550** is operating (e.g., when the vehicle is in motion). The steering system **554** may receive signals from a steering actuator **556**. The steering wheel may be optional for full automation (Level 5) functionality.

[0064] The brake sensor system **546** may be used to operate the vehicle brakes in response to receiving signals from the brake actuators **548** and/or brake sensors.

[0065] Controller(s) **536**, which may include one or more system on chips (SoCs) **504** (FIG. 5C) and/or GPU(s), may provide signals (e.g., representative of commands) to one or more components and/or systems of the vehicle **500**. For example, the controller(s) may send signals to operate the vehicle brakes via one or more brake actuators **548**, to operate the steering system **554** via one or more steering actuators **556**, to operate the propulsion system **550** via one or more throttle/accelerators **552**. The controller(s) **536** may include one or more onboard (e.g., integrated) computing devices (e.g., supercomputers) that process sensor signals, and output operation commands (e.g., signals representing commands) to enable autonomous driving and/or to assist a human driver in driving the vehicle **500**. The controller(s) **536** may include a first controller **536** for autonomous driving functions, a second controller **536** for functional safety functions, a third controller **536** for artificial intelligence functionality (e.g., computer vision), a fourth controller **536** for infotainment functionality, a fifth controller **536** for redundancy in emergency conditions,

and/or other controllers. In some examples, a single controller **536** may handle two or more of the above functionalities, two or more controllers **536** may handle a single functionality, and/or any combination thereof.

[0066] The controller(s) **536** may provide the signals for controlling one or more components and/or systems of the vehicle **500** in response to sensor data received from one or more sensors (e.g., sensor inputs). The sensor data may be received from, for example and without limitation, global navigation satellite systems sensor(s) **558** (e.g., Global Positioning System sensor(s)), RADAR sensor(s) **560**, ultrasonic sensor(s) **562**, LIDAR sensor(s) **564**, inertial measurement unit (IMU) sensor(s) **566** (e.g., accelerometer(s), gyroscope(s), magnetic compass(es), magnetometer(s), etc.), microphone(s) **596**, stereo camera(s) **568**, wide-view camera(s) **570** (e.g., fisheye cameras), infrared camera(s) **572**, surround camera(s) **574** (e.g., 360 degree cameras), long-range and/or mid-range camera(s) **598**, speed sensor(s) **544** (e.g., for measuring the speed of the vehicle **500**), vibration sensor(s) **542**, steering sensor(s) **540**, brake sensor(s) (e.g., as part of the brake sensor system **546**), and/or other sensor types.

[0067] One or more of the controller(s) **536** may receive inputs (e.g., represented by input data) from an instrument cluster **532** of the vehicle **500** and provide outputs (e.g., represented by output data, display data, etc.) via a human-machine interface (HMI) display **534**, an audible annunciator, a loudspeaker, and/or via other components of the vehicle **500**. The outputs may include information such as vehicle velocity, speed, time, map data (e.g., the HD map **522** of FIG. 5C), location data (e.g., the vehicle's **500** location, such as on a map), direction, location of other vehicles (e.g., an occupancy grid), information about objects and status of objects as perceived by the controller(s) **536**, etc. For example, the HMI display **534** may display information about the presence of one or more objects (e.g., a street sign, caution sign, traffic light changing, etc.), and/or information about driving maneuvers the vehicle has made, is making, or will make (e.g., changing lanes now, taking exit **34B** in two miles, etc.).

[0068] The vehicle **500** further includes a network interface **524** which may use one or more wireless antenna(s) **526** and/or modem(s) to communicate over one or more networks. For example, the network interface **524** may be capable of communication over LTE, WCDMA, UMTS, GSM, CDMA2000, etc. The wireless antenna(s) **526** may also enable communication between objects in the environment (e.g., vehicles, mobile devices, etc.), using local area network(s), such as Bluetooth, Bluetooth LE, Z-Wave, ZigBee, etc., and/or low power wide-area network(s) (LPWANs), such as LoRaWAN, SigFox, etc.

[0069] FIG. 5B is an example of camera locations and fields of view for the example autonomous vehicle **500** of FIG. 5A, in accordance with some embodiments of the present disclosure. The cameras and respective fields of view are one example embodiment and are not intended to be limiting. For example, additional and/or alternative cameras may be included and/or the cameras may be located at different locations on the vehicle **500**.

[0070] The camera types for the cameras may include, but are not limited to, digital cameras that may be adapted for use with the components and/or systems of the vehicle **500**. The camera(s) may operate at automotive safety integrity level (ASIL) B and/or at another ASIL. The camera types may be capable of any image capture rate, such as 60 frames per second (fps), 120 fps, 240 fps, etc., depending on the embodiment. The cameras may be capable of using rolling shutters, global shutters, another type of shutter, or a combination thereof. In some examples, the color filter array may include a red clear clear clear (RCCC) color filter array, a red clear clear blue (RCCB) color filter array, a red blue green clear (RBGC) color filter array, a Foveon X3 color filter array, a Bayer sensors (RGGB) color filter array, a monochrome sensor color filter array, and/or another type of color filter array. In some embodiments, clear pixel cameras, such as cameras with an RCCC, an RCCB, and/or an RBGC color filter array, may be used in an effort to increase light sensitivity.

[0071] In some examples, one or more of the camera(s) may be used to perform advanced driver assistance systems (ADAS) functions (e.g., as part of a redundant or fail-safe design). For example,

a Multi-Function Mono Camera may be installed to provide functions including lane departure warning, traffic sign assist and intelligent headlamp control. One or more of the camera(s) (e.g., all of the cameras) may record and provide image data (e.g., video) simultaneously.

[0072] One or more of the cameras may be mounted in a mounting assembly, such as a custom designed (3-D printed) assembly, in order to cut out stray light and reflections from within the car (e.g., reflections from the dashboard reflected in the windshield mirrors) which may interfere with the camera's image data capture abilities. With reference to wing-mirror mounting assemblies, the wing-mirror assemblies may be custom 3-D printed so that the camera mounting plate matches the shape of the wing-mirror. In some examples, the camera(s) may be integrated into the wing-mirror. For side-view cameras, the camera(s) may also be integrated within the four pillars at each corner of the cabin.

[0073] Cameras with a field of view that include portions of the environment in front of the vehicle **500** (e.g., front-facing cameras) may be used for surround view, to help identify forward facing paths and obstacles, as well aid in, with the help of one or more controllers **536** and/or control SoCs, providing information critical to generating an occupancy grid and/or determining the preferred vehicle paths. Front-facing cameras may be used to perform many of the same ADAS functions as LIDAR, including emergency braking, pedestrian detection, and collision avoidance. Front-facing cameras may also be used for ADAS functions and systems including Lane Departure Warnings (LDW), Autonomous Cruise Control (ACC), and/or other functions such as traffic sign recognition.

[0074] A variety of cameras may be used in a front-facing configuration, including, for example, a monocular camera platform that includes a CMOS (complementary metal oxide semiconductor) color imager. Another example may be a wide-view camera(s) **570** that may be used to perceive objects coming into view from the periphery (e.g., pedestrians, crossing traffic or bicycles). Although only one wide-view camera is illustrated in FIG. 5B, there may any number of wide-view cameras **570** on the vehicle **500**. In addition, long-range camera(s) **598** (e.g., a long-view stereo camera pair) may be used for depth-based object detection, especially for objects for which a neural network has not yet been trained. The long-range camera(s) **598** may also be used for object detection and classification, as well as basic object tracking.

[0075] One or more stereo cameras **568** may also be included in a front-facing configuration. The stereo camera(s) **568** may include an integrated control unit comprising a scalable processing unit, which may provide a programmable logic (FPGA) and a multi-core micro-processor with an integrated CAN or Ethernet interface on a single chip. Such a unit may be used to generate a 3-D map of the vehicle's environment, including a distance estimate for all the points in the image. An alternative stereo camera(s) **568** may include a compact stereo vision sensor(s) that may include two camera lenses (one each on the left and right) and an image processing chip that may measure the distance from the vehicle to the target object and use the generated information (e.g., metadata) to activate the autonomous emergency braking and lane departure warning functions. Other types of stereo camera(s) **568** may be used in addition to, or alternatively from, those described herein.

[0076] Cameras with a field of view that include portions of the environment to the side of the vehicle **500** (e.g., side-view cameras) may be used for surround view, providing information used to create and update the occupancy grid, as well as to generate side impact collision warnings. For example, surround camera(s) **574** (e.g., four surround cameras **574** as illustrated in FIG. 5B) may be positioned to on the vehicle **500**. The surround camera(s) **574** may include wide-view camera(s) **570**, fisheye camera(s), 360 degree camera(s), and/or the like. Four example, four fisheye cameras may be positioned on the vehicle's front, rear, and sides. In an alternative arrangement, the vehicle may use three surround camera(s) **574** (e.g., left, right, and rear), and may leverage one or more other camera(s) (e.g., a forward-facing camera) as a fourth surround view camera.

[0077] Cameras with a field of view that include portions of the environment to the rear of the vehicle **500** (e.g., rear-view cameras) may be used for park assistance, surround view, rear collision

warnings, and creating and updating the occupancy grid. A wide variety of cameras may be used including, but not limited to, cameras that are also suitable as a front-facing camera(s) (e.g., long-range and/or mid-range camera(s) **598**, stereo camera(s) **568**), infrared camera(s) **572**, etc.), as described herein.

[0078] FIG. 5C is a block diagram of an example system architecture for the example autonomous vehicle **500** of FIG. 5A, in accordance with some embodiments of the present disclosure. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces, functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory.

[0079] Each of the components, features, and systems of the vehicle **500** in FIG. 5C are illustrated as being connected via bus **502**. The bus **502** may include a Controller Area Network (CAN) data interface (alternatively referred to herein as a “CAN bus”). A CAN may be a network inside the vehicle **500** used to aid in control of various features and functionality of the vehicle **500**, such as actuation of brakes, acceleration, braking, steering, windshield wipers, etc. A CAN bus may be configured to have dozens or even hundreds of nodes, each with its own unique identifier (e.g., a CAN ID). The CAN bus may be read to find steering wheel angle, ground speed, engine revolutions per minute (RPMs), button positions, and/or other vehicle status indicators. The CAN bus may be ASIL B compliant.

[0080] Although the bus **502** is described herein as being a CAN bus, this is not intended to be limiting. For example, in addition to, or alternatively from, the CAN bus, FlexRay and/or Ethernet may be used. Additionally, although a single line is used to represent the bus **502**, this is not intended to be limiting. For example, there may be any number of busses **502**, which may include one or more CAN busses, one or more FlexRay busses, one or more Ethernet busses, and/or one or more other types of busses using a different protocol. In some examples, two or more busses **502** may be used to perform different functions, and/or may be used for redundancy. For example, a first bus **502** may be used for collision avoidance functionality and a second bus **502** may be used for actuation control. In any example, each bus **502** may communicate with any of the components of the vehicle **500**, and two or more busses **502** may communicate with the same components. In some examples, each SoC **504**, each controller **536**, and/or each computer within the vehicle may have access to the same input data (e.g., inputs from sensors of the vehicle **500**), and may be connected to a common bus, such the CAN bus.

[0081] The vehicle **500** may include one or more controller(s) **536**, such as those described herein with respect to FIG. 5A. The controller(s) **536** may be used for a variety of functions. The controller(s) **536** may be coupled to any of the various other components and systems of the vehicle **500**, and may be used for control of the vehicle **500**, artificial intelligence of the vehicle **500**, infotainment for the vehicle **500**, and/or the like.

[0082] The vehicle **500** may include a system(s) on a chip (SoC) **504**. The SoC **504** may include CPU(s) **506**, GPU(s) **508**, processor(s) **510**, cache(s) **512**, accelerator(s) **514**, data store(s) **516**, and/or other components and features not illustrated. The SoC(s) **504** may be used to control the vehicle **500** in a variety of platforms and systems. For example, the SoC(s) **504** may be combined in a system (e.g., the system of the vehicle **500**) with an HD map **522** which may obtain map refreshes and/or updates via a network interface **524** from one or more servers (e.g., server(s) **578** of FIG. 5D).

[0083] The CPU(s) **506** may include a CPU cluster or CPU complex (alternatively referred to herein as a “CCPLEX”). The CPU(s) **506** may include multiple cores and/or L2 caches. For

example, in some embodiments, the CPU(s) **506** may include eight cores in a coherent multi-processor configuration. In some embodiments, the CPU(s) **506** may include four dual-core clusters where each cluster has a dedicated L2 cache (e.g., a 2 MB L2 cache). The CPU(s) **506** (e.g., the CCPLEX) may be configured to support simultaneous cluster operation enabling any combination of the clusters of the CPU(s) **506** to be active at any given time.

[0084] The CPU(s) **506** may implement power management capabilities that include one or more of the following features: individual hardware blocks may be clock-gated automatically when idle to save dynamic power; each core clock may be gated when the core is not actively executing instructions due to execution of WFI/WFE instructions; each core may be independently power-gated; each core cluster may be independently clock-gated when all cores are clock-gated or power-gated; and/or each core cluster may be independently power-gated when all cores are power-gated. The CPU(s) **506** may further implement an enhanced algorithm for managing power states, where allowed power states and expected wakeup times are specified, and the hardware/microcode determines the best power state to enter for the core, cluster, and CCPLEX. The processing cores may support simplified power state entry sequences in software with the work offloaded to microcode.

[0085] The GPU(s) **508** may include an integrated GPU (alternatively referred to herein as an “iGPU”). The GPU(s) **508** may be programmable and may be efficient for parallel workloads. The GPU(s) **508**, in some examples, may use an enhanced tensor instruction set. The GPU(s) **508** may include one or more streaming microprocessors, where each streaming microprocessor may include an L1 cache (e.g., an L1 cache with at least 96 KB storage capacity), and two or more of the streaming microprocessors may share an L2 cache (e.g., an L2 cache with a 512 KB storage capacity). In some embodiments, the GPU(s) **508** may include at least eight streaming microprocessors. The GPU(s) **508** may use compute application programming interface(s) (API(s)). In addition, the GPU(s) **508** may use one or more parallel computing platforms and/or programming models (e.g., NVIDIA's CUDA).

[0086] The GPU(s) **508** may be power-optimized for best performance in automotive and embedded use cases. For example, the GPU(s) **508** may be fabricated on a Fin field-effect transistor (FinFET). However, this is not intended to be limiting and the GPU(s) **508** may be fabricated using other semiconductor manufacturing processes. Each streaming microprocessor may incorporate a number of mixed-precision processing cores partitioned into multiple blocks. For example, and without limitation, 64 PF32 cores and 32 PF64 cores may be partitioned into four processing blocks. In such an example, each processing block may be allocated 16 FP32 cores, 8 FP64 cores, 16 INT32 cores, two mixed-precision NVIDIA TENSOR COREs for deep learning matrix arithmetic, an L0 instruction cache, a warp scheduler, a dispatch unit, and/or a 64 KB register file. In addition, the streaming microprocessors may include independent parallel integer and floating-point data paths to provide for efficient execution of workloads with a mix of computation and addressing calculations. The streaming microprocessors may include independent thread scheduling capability to enable finer-grain synchronization and cooperation between parallel threads. The streaming microprocessors may include a combined L1 data cache and shared memory unit in order to improve performance while simplifying programming.

[0087] The GPU(s) **508** may include a high bandwidth memory (HBM) and/or a 16 GB HBM2 memory subsystem to provide, in some examples, about 900 GB/second peak memory bandwidth. In some examples, in addition to, or alternatively from, the HBM memory, a synchronous graphics random-access memory (SGRAM) may be used, such as a graphics double data rate type five synchronous random-access memory (GDDR5).

[0088] The GPU(s) **508** may include unified memory technology including access counters to allow for more accurate migration of memory pages to the processor that accesses them most frequently, thereby improving efficiency for memory ranges shared between processors. In some examples, address translation services (ATS) support may be used to allow the GPU(s) **508** to

access the CPU(s) 506 page tables directly. In such examples, when the GPU(s) 508 memory management unit (MMU) experiences a miss, an address translation request may be transmitted to the CPU(s) 506. In response, the CPU(s) 506 may look in its page tables for the virtual-to-physical mapping for the address and transmits the translation back to the GPU(s) 508. As such, unified memory technology may allow a single unified virtual address space for memory of both the CPU(s) 506 and the GPU(s) 508, thereby simplifying the GPU(s) 508 programming and porting of applications to the GPU(s) 508.

[0089] In addition, the GPU(s) 508 may include an access counter that may keep track of the frequency of access of the GPU(s) 508 to memory of other processors. The access counter may help ensure that memory pages are moved to the physical memory of the processor that is accessing the pages most frequently.

[0090] The SoC(s) 504 may include any number of cache(s) 512, including those described herein. For example, the cache(s) 512 may include an L3 cache that is available to both the CPU(s) 506 and the GPU(s) 508 (e.g., that is connected both the CPU(s) 506 and the GPU(s) 508). The cache(s) 512 may include a write-back cache that may keep track of states of lines, such as by using a cache coherence protocol (e.g., MEI, MESI, MSI, etc.). The L3 cache may include 4 MB or more, depending on the embodiment, although smaller cache sizes may be used.

[0091] The SoC(s) 504 may include an arithmetic logic unit(s) (ALU(s)) which may be leveraged in performing processing with respect to any of the variety of tasks or operations of the vehicle 500—such as processing DNNs. In addition, the SoC(s) 504 may include a floating point unit(s) (FPU(s))—or other math coprocessor or numeric coprocessor types—for performing mathematical operations within the system. For example, the SoC(s) 104 may include one or more FPUs integrated as execution units within a CPU(s) 506 and/or GPU(s) 508.

[0092] The SoC(s) 504 may include one or more accelerators 514 (e.g., hardware accelerators, software accelerators, or a combination thereof). For example, the SoC(s) 504 may include a hardware acceleration cluster that may include optimized hardware accelerators and/or large on-chip memory. The large on-chip memory (e.g., 4 MB of SRAM), may enable the hardware acceleration cluster to accelerate neural networks and other calculations. The hardware acceleration cluster may be used to complement the GPU(s) 508 and to off-load some of the tasks of the GPU(s) 508 (e.g., to free up more cycles of the GPU(s) 508 for performing other tasks). As an example, the accelerator(s) 514 may be used for targeted workloads (e.g., perception, convolutional neural networks (CNNs), etc.) that are stable enough to be amenable to acceleration. The term “CNN,” as used herein, may include all types of CNNs, including region-based or regional convolutional neural networks (RCNNs) and Fast RCNNs (e.g., as used for object detection).

[0093] The accelerator(s) 514 (e.g., the hardware acceleration cluster) may include a deep learning accelerator(s) (DLA). The DLA(s) may include one or more Tensor processing units (TPUs) that may be configured to provide an additional ten trillion operations per second for deep learning applications and inferencing. The TPUs may be accelerators configured to, and optimized for, performing image processing functions (e.g., for CNNs, RCNNs, etc.). The DLA(s) may further be optimized for a specific set of neural network types and floating point operations, as well as inferencing. The design of the DLA(s) may provide more performance per millimeter than a general-purpose GPU, and vastly exceeds the performance of a CPU. The TPU(s) may perform several functions, including a single-instance convolution function, supporting, for example, INT8, INT16, and FP16 data types for both features and weights, as well as post-processor functions.

[0094] The DLA(s) may quickly and efficiently execute neural networks, especially CNNs, on processed or unprocessed data for any of a variety of functions, including, for example and without limitation: a CNN for object identification and detection using data from camera sensors; a CNN for distance estimation using data from camera sensors; a CNN for emergency vehicle detection and identification and detection using data from microphones; a CNN for facial recognition and vehicle owner identification using data from camera sensors; and/or a CNN for security and/or

safety related events.

[0095] The DLA(s) may perform any function of the GPU(s) **508**, and by using an inference accelerator, for example, a designer may target either the DLA(s) or the GPU(s) **508** for any function. For example, the designer may focus processing of CNNs and floating point operations on the DLA(s) and leave other functions to the GPU(s) **508** and/or other accelerator(s) **514**.

[0096] The accelerator(s) **514** (e.g., the hardware acceleration cluster) may include a programmable vision accelerator(s) (PVA), which may alternatively be referred to herein as a computer vision accelerator. The PVA(s) may be designed and configured to accelerate computer vision algorithms for the advanced driver assistance systems (ADAS), autonomous driving, and/or augmented reality (AR) and/or virtual reality (VR) applications. The PVA(s) may provide a balance between performance and flexibility. For example, each PVA(s) may include, for example and without limitation, any number of reduced instruction set computer (RISC) cores, direct memory access (DMA), and/or any number of vector processors.

[0097] The RISC cores may interact with image sensors (e.g., the image sensors of any of the cameras described herein), image signal processor(s), and/or the like. Each of the RISC cores may include any amount of memory. The RISC cores may use any of a number of protocols, depending on the embodiment. In some examples, the RISC cores may execute a real-time operating system (RTOS). The RISC cores may be implemented using one or more integrated circuit devices, application specific integrated circuits (ASICs), and/or memory devices. For example, the RISC cores may include an instruction cache and/or a tightly coupled RAM.

[0098] The DMA may enable components of the PVA(s) to access the system memory independently of the CPU(s) **506**. The DMA may support any number of features used to provide optimization to the PVA including, but not limited to, supporting multi-dimensional addressing and/or circular addressing. In some examples, the DMA may support up to six or more dimensions of addressing, which may include block width, block height, block depth, horizontal block stepping, vertical block stepping, and/or depth stepping.

[0099] The vector processors may be programmable processors that may be designed to efficiently and flexibly execute programming for computer vision algorithms and provide signal processing capabilities. In some examples, the PVA may include a PVA core and two vector processing subsystem partitions. The PVA core may include a processor subsystem, DMA engine(s) (e.g., two DMA engines), and/or other peripherals. The vector processing subsystem may operate as the primary processing engine of the PVA, and may include a vector processing unit (VPU), an instruction cache, and/or vector memory (e.g., VMEM). A VPU core may include a digital signal processor such as, for example, a single instruction, multiple data (SIMD), very long instruction word (VLIW) digital signal processor. The combination of the SIMD and VLIW may enhance throughput and speed.

[0100] Each of the vector processors may include an instruction cache and may be coupled to dedicated memory. As a result, in some examples, each of the vector processors may be configured to execute independently of the other vector processors. In other examples, the vector processors that are included in a particular PVA may be configured to employ data parallelism. For example, in some embodiments, the plurality of vector processors included in a single PVA may execute the same computer vision algorithm, but on different regions of an image. In other examples, the vector processors included in a particular PVA may simultaneously execute different computer vision algorithms, on the same image, or even execute different algorithms on sequential images or portions of an image. Among other things, any number of PVAs may be included in the hardware acceleration cluster and any number of vector processors may be included in each of the PVAs. In addition, the PVA(s) may include additional error correcting code (ECC) memory, to enhance overall system safety.

[0101] The accelerator(s) **514** (e.g., the hardware acceleration cluster) may include a computer vision network on-chip and SRAM, for providing a high-bandwidth, low latency SRAM for the

accelerator(s) **514**. In some examples, the on-chip memory may include at least 4 MB SRAM, consisting of, for example and without limitation, eight field-configurable memory blocks, that may be accessible by both the PVA and the DLA. Each pair of memory blocks may include an advanced peripheral bus (APB) interface, configuration circuitry, a controller, and a multiplexer. Any type of memory may be used. The PVA and DLA may access the memory via a backbone that provides the PVA and DLA with high-speed access to memory. The backbone may include a computer vision network on-chip that interconnects the PVA and the DLA to the memory (e.g., using the APB).

[0102] The computer vision network on-chip may include an interface that determines, before transmission of any control signal/address/data, that both the PVA and the DLA provide ready and valid signals. Such an interface may provide for separate phases and separate channels for transmitting control signals/addresses/data, as well as burst-type communications for continuous data transfer. This type of interface may comply with ISO 26262 or IEC 61508 standards, although other standards and protocols may be used.

[0103] In some examples, the SoC(s) **504** may include a real-time ray-tracing hardware accelerator, such as described in U.S. patent application Ser. No. 16/101,232, filed on Aug. 10, 2018. The real-time ray-tracing hardware accelerator may be used to quickly and efficiently determine the positions and extents of objects (e.g., within a world model), to generate real-time visualization simulations, for RADAR signal interpretation, for sound propagation synthesis and/or analysis, for simulation of SONAR systems, for general wave propagation simulation, for comparison to LIDAR data for purposes of localization and/or other functions, and/or for other uses. In some embodiments, one or more tree traversal units (TTUs) may be used for executing one or more ray-tracing related operations.

[0104] The accelerator(s) **514** (e.g., the hardware accelerator cluster) have a wide array of uses for autonomous driving. The PVA may be a programmable vision accelerator that may be used for key processing stages in ADAS and autonomous vehicles. The PVA's capabilities are a good match for algorithmic domains needing predictable processing, at low power and low latency. In other words, the PVA performs well on semi-dense or dense regular computation, even on small data sets, which need predictable run-times with low latency and low power. Thus, in the context of platforms for autonomous vehicles, the PVAs are designed to run classic computer vision algorithms, as they are efficient at object detection and operating on integer math.

[0105] For example, according to one embodiment of the technology, the PVA is used to perform computer stereo vision. A semi-global matching-based algorithm may be used in some examples, although this is not intended to be limiting. Many applications for Level 3-5 autonomous driving require motion estimation/stereo matching on-the-fly (e.g., structure from motion, pedestrian recognition, lane detection, etc.). The PVA may perform computer stereo vision function on inputs from two monocular cameras.

[0106] In some examples, the PVA may be used to perform dense optical flow. According to process raw RADAR data (e.g., using a 4D Fast Fourier Transform) to provide Processed RADAR. In other examples, the PVA is used for time of flight depth processing, by processing raw time of flight data to provide processed time of flight data, for example.

[0107] The DLA may be used to run any type of network to enhance control and driving safety, including for example, a neural network that outputs a measure of confidence for each object detection. Such a confidence value may be interpreted as a probability, or as providing a relative "weight" of each detection compared to other detections. This confidence value enables the system to make further decisions regarding which detections should be considered as true positive detections rather than false positive detections. For example, the system may set a threshold value for the confidence and consider only the detections exceeding the threshold value as true positive detections. In an automatic emergency braking (AEB) system, false positive detections would cause the vehicle to automatically perform emergency braking, which is obviously undesirable.

Therefore, only the most confident detections should be considered as triggers for AEB. The DLA may run a neural network for regressing the confidence value. The neural network may take as its input at least some subset of parameters, such as bounding box dimensions, ground plane estimate obtained (e.g. from another subsystem), inertial measurement unit (IMU) sensor **566** output that correlates with the vehicle **500** orientation, distance, 3D location estimates of the object obtained from the neural network and/or other sensors (e.g., LIDAR sensor(s) **564** or RADAR sensor(s) **560**), among others.

[0108] The SoC(s) **504** may include data store(s) **516** (e.g., memory). The data store(s) **516** may be on-chip memory of the SoC(s) **504**, which may store neural networks to be executed on the GPU and/or the DLA. In some examples, the data store(s) **516** may be large enough in capacity to store multiple instances of neural networks for redundancy and safety. The data store(s) **512** may comprise L2 or L3 cache(s) **512**. Reference to the data store(s) **516** may include reference to the memory associated with the PVA, DLA, and/or other accelerator(s) **514**, as described herein.

[0109] The SoC(s) **504** may include one or more processor(s) **510** (e.g., embedded processors). The processor(s) **510** may include a boot and power management processor that may be a dedicated processor and subsystem to handle boot power and management functions and related security enforcement. The boot and power management processor may be a part of the SoC(s) **504** boot sequence and may provide runtime power management services. The boot power and management processor may provide clock and voltage programming, assistance in system low power state transitions, management of SoC(s) **504** thermals and temperature sensors, and/or management of the SoC(s) **504** power states. Each temperature sensor may be implemented as a ring-oscillator whose output frequency is proportional to temperature, and the SoC(s) **504** may use the ring-oscillators to detect temperatures of the CPU(s) **506**, GPU(s) **508**, and/or accelerator(s) **514**. If temperatures are determined to exceed a threshold, the boot and power management processor may enter a temperature fault routine and put the SoC(s) **504** into a lower power state and/or put the vehicle **500** into a chauffeur to safe stop mode (e.g., bring the vehicle **500** to a safe stop).

[0110] The processor(s) **510** may further include a set of embedded processors that may serve as an audio processing engine. The audio processing engine may be an audio subsystem that enables full hardware support for multi-channel audio over multiple interfaces, and a broad and flexible range of audio I/O interfaces. In some examples, the audio processing engine is a dedicated processor core with a digital signal processor with dedicated RAM.

[0111] The processor(s) **510** may further include an always on processor engine that may provide necessary hardware features to support low power sensor management and wake use cases. The always on processor engine may include a processor core, a tightly coupled RAM, supporting peripherals (e.g., timers and interrupt controllers), various I/O controller peripherals, and routing logic.

[0112] The processor(s) **510** may further include a safety cluster engine that includes a dedicated processor subsystem to handle safety management for automotive applications. The safety cluster engine may include two or more processor cores, a tightly coupled RAM, support peripherals (e.g., timers, an interrupt controller, etc.), and/or routing logic. In a safety mode, the two or more cores may operate in a lockstep mode and function as a single core with comparison logic to detect any differences between their operations.

[0113] The processor(s) **510** may further include a real-time camera engine that may include a dedicated processor subsystem for handling real-time camera management.

[0114] The processor(s) **510** may further include a high-dynamic range signal processor that may include an image signal processor that is a hardware engine that is part of the camera processing pipeline.

[0115] The processor(s) **510** may include a video image compositor that may be a processing block (e.g., implemented on a microprocessor) that implements video post-processing functions needed by a video playback application to produce the final image for the player window. The video image

compositor may perform lens distortion correction on wide-view camera(s) **570**, surround camera(s) **574**, and/or on in-cabin monitoring camera sensors. In-cabin monitoring camera sensor is preferably monitored by a neural network running on another instance of the Advanced SoC, configured to identify in cabin events and respond accordingly. An in-cabin system may perform lip reading to activate cellular service and place a phone call, dictate emails, change the vehicle's destination, activate or change the vehicle's infotainment system and settings, or provide voice-activated web surfing. Certain functions are available to the driver only when the vehicle is operating in an autonomous mode, and are disabled otherwise.

[0116] The video image compositor may include enhanced temporal noise reduction for both spatial and temporal noise reduction. For example, where motion occurs in a video, the noise reduction weights spatial information appropriately, decreasing the weight of information provided by adjacent frames. Where an image or portion of an image does not include motion, the temporal noise reduction performed by the video image compositor may use information from the previous image to reduce noise in the current image.

[0117] The video image compositor may also be configured to perform stereo rectification on input stereo lens frames. The video image compositor may further be used for user interface composition when the operating system desktop is in use, and the GPU(s) **508** is not required to continuously render new surfaces. Even when the GPU(s) **508** is powered on and active doing 3D rendering, the video image compositor may be used to offload the GPU(s) **508** to improve performance and responsiveness.

[0118] The SoC(s) **504** may further include a mobile industry processor interface (MIPI) camera serial interface for receiving video and input from cameras, a high-speed interface, and/or a video input block that may be used for camera and related pixel input functions. The SoC(s) **504** may further include an input/output controller(s) that may be controlled by software and may be used for receiving I/O signals that are uncommitted to a specific role.

[0119] The SoC(s) **504** may further include a broad range of peripheral interfaces to enable communication with peripherals, audio codecs, power management, and/or other devices. The SoC(s) **504** may be used to process data from cameras (e.g., connected over Gigabit Multimedia Serial Link and Ethernet), sensors (e.g., LIDAR sensor(s) **564**, RADAR sensor(s) **560**, etc. that may be connected over Ethernet), data from bus **502** (e.g., speed of vehicle **500**, steering wheel position, etc.), data from GNSS sensor(s) **558** (e.g., connected over Ethernet or CAN bus). The SoC(s) **504** may further include dedicated high-performance mass storage controllers that may include their own DMA engines, and that may be used to free the CPU(s) **506** from routine data management tasks.

[0120] The SoC(s) **504** may be an end-to-end platform with a flexible architecture that spans automation levels 3-5, thereby providing a comprehensive functional safety architecture that leverages and makes efficient use of computer vision and ADAS techniques for diversity and redundancy, provides a platform for a flexible, reliable driving software stack, along with deep learning tools. The SoC(s) **504** may be faster, more reliable, and even more energy-efficient and space-efficient than conventional systems. For example, the accelerator(s) **514**, when combined with the CPU(s) **506**, the GPU(s) **508**, and the data store(s) **516**, may provide for a fast, efficient platform for level 3-5 autonomous vehicles.

[0121] The technology thus provides capabilities and functionality that cannot be achieved by conventional systems. For example, computer vision algorithms may be executed on CPUs, which may be configured using high-level programming language, such as the C programming language, to execute a wide variety of processing algorithms across a wide variety of visual data. However, CPUs are oftentimes unable to meet the performance requirements of many computer vision applications, such as those related to execution time and power consumption, for example. In particular, many CPUs are unable to execute complex object detection algorithms in real-time, which is a requirement of in-vehicle ADAS applications, and a requirement for practical Level 3-5

autonomous vehicles.

[0122] In contrast to conventional systems, by providing a CPU complex, GPU complex, and a hardware acceleration cluster, the technology described herein allows for multiple neural networks to be performed simultaneously and/or sequentially, and for the results to be combined together to enable Level 3-5 autonomous driving functionality. For example, a CNN executing on the DLA or dGPU (e.g., the GPU(s) **520**) may include a text and word recognition, allowing the supercomputer to read and understand traffic signs, including signs for which the neural network has not been specifically trained. The DLA may further include a neural network that is able to identify, interpret, and provides semantic understanding of the sign, and to pass that semantic understanding to the path planning modules running on the CPU Complex.

[0123] As another example, multiple neural networks may be run simultaneously, as is required for Level 3, 4, or 5 driving. For example, a warning sign consisting of “Caution: flashing lights indicate icy conditions,” along with an electric light, may be independently or collectively interpreted by several neural networks. The sign itself may be identified as a traffic sign by a first deployed neural network (e.g., a neural network that has been trained), the text “Flashing lights indicate icy conditions” may be interpreted by a second deployed neural network, which informs the vehicle's path planning software (preferably executing on the CPU Complex) that when flashing lights are detected, icy conditions exist. The flashing light may be identified by operating a third deployed neural network over multiple frames, informing the vehicle's path-planning software of the presence (or absence) of flashing lights. All three neural networks may run simultaneously, such as within the DLA and/or on the GPU(s) **508**.

[0124] In some examples, a CNN for facial recognition and vehicle owner identification may use data from camera sensors to identify the presence of an authorized driver and/or owner of the vehicle **500**. The always on sensor processing engine may be used to unlock the vehicle when the owner approaches the driver door and turn on the lights, and, in security mode, to disable the vehicle when the owner leaves the vehicle. In this way, the SoC(s) **504** provide for security against theft and/or carjacking.

[0125] In another example, a CNN for emergency vehicle detection and identification may use data from microphones **596** to detect and identify emergency vehicle sirens. In contrast to conventional systems, that use general classifiers to detect sirens and manually extract features, the SoC(s) **504** use the CNN for classifying environmental and urban sounds, as well as classifying visual data. In a preferred embodiment, the CNN running on the DLA is trained to identify the relative closing speed of the emergency vehicle (e.g., by using the Doppler Effect). The CNN may also be trained to identify emergency vehicles specific to the local area in which the vehicle is operating, as identified by GNSS sensor(s) **558**. Thus, for example, when operating in Europe the CNN will seek to detect European sirens, and when in the United States the CNN will seek to identify only North American sirens. Once an emergency vehicle is detected, a control program may be used to execute an emergency vehicle safety routine, slowing the vehicle, pulling over to the side of the road, parking the vehicle, and/or idling the vehicle, with the assistance of ultrasonic sensors **562**, until the emergency vehicle(s) passes.

[0126] The vehicle may include a CPU(s) **518** (e.g., discrete CPU(s), or dCPU(s)), that may be coupled to the SoC(s) **504** via a high-speed interconnect (e.g., PCIe). The CPU(s) **518** may include an X86 processor, for example. The CPU(s) **518** may be used to perform any of a variety of functions, including arbitrating potentially inconsistent results between ADAS sensors and the SoC(s) **504**, and/or monitoring the status and health of the controller(s) **536** and/or infotainment SoC **530**, for example.

[0127] The vehicle **500** may include a GPU(s) **520** (e.g., discrete GPU(s), or dGPU(s)), that may be coupled to the SoC(s) **504** via a high-speed interconnect (e.g., NVIDIA's NVLINK). The GPU(s) **520** may provide additional artificial intelligence functionality, such as by executing redundant and/or different neural networks, and may be used to train and/or update neural networks based on

input (e.g., sensor data) from sensors of the vehicle **500**.

[0128] The vehicle **500** may further include the network interface **524** which may include one or more wireless antennas **526** (e.g., one or more wireless antennas for different communication protocols, such as a cellular antenna, a Bluetooth antenna, etc.). The network interface **524** may be used to enable wireless connectivity over the Internet with the cloud (e.g., with the server(s) **578** and/or other network devices), with other vehicles, and/or with computing devices (e.g., client devices of passengers). To communicate with other vehicles, a direct link may be established between the two vehicles and/or an indirect link may be established (e.g., across networks and over the Internet). Direct links may be provided using a vehicle-to-vehicle communication link. The vehicle-to-vehicle communication link may provide the vehicle **500** information about vehicles in proximity to the vehicle **500** (e.g., vehicles in front of, on the side of, and/or behind the vehicle **500**). This functionality may be part of a cooperative adaptive cruise control functionality of the vehicle **500**.

[0129] The network interface **524** may include a SoC that provides modulation and demodulation functionality and enables the controller(s) **536** to communicate over wireless networks. The network interface **524** may include a radio frequency front-end for up-conversion from baseband to radio frequency, and down conversion from radio frequency to baseband. The frequency conversions may be performed through well-known processes, and/or may be performed using super-heterodyne processes. In some examples, the radio frequency front end functionality may be provided by a separate chip. The network interface may include wireless functionality for communicating over LTE, WCDMA, UMTS, GSM, CDMA2000, Bluetooth, Bluetooth LE, Wi-Fi, Z-Wave, ZigBee, LoRaWAN, and/or other wireless protocols.

[0130] The vehicle **500** may further include data store(s) **528** which may include off-chip (e.g., off the SoC(s) **504**) storage. The data store(s) **528** may include one or more storage elements including RAM, SRAM, DRAM, VRAM, Flash, hard disks, and/or other components and/or devices that may store at least one bit of data.

[0131] The vehicle **500** may further include GNSS sensor(s) **558**. The GNSS sensor(s) **558** (e.g., GPS, assisted GPS sensors, differential GPS (DGPS) sensors, etc.), to assist in mapping, perception, occupancy grid generation, and/or path planning functions. Any number of GNSS sensor(s) **558** may be used, including, for example and without limitation, a GPS using a USB connector with an Ethernet to Serial (RS-232) bridge.

[0132] The vehicle **500** may further include RADAR sensor(s) **560**. The RADAR sensor(s) **560** may be used by the vehicle **500** for long-range vehicle detection, even in darkness and/or severe weather conditions. RADAR functional safety levels may be ASIL B. The RADAR sensor(s) **560** may use the CAN and/or the bus **502** (e.g., to transmit data generated by the RADAR sensor(s) **560**) for control and to access object tracking data, with access to Ethernet to access raw data in some examples. A wide variety of RADAR sensor types may be used. For example, and without limitation, the RADAR sensor(s) **560** may be suitable for front, rear, and side RADAR use. In some example, Pulse Doppler RADAR sensor(s) are used.

[0133] The RADAR sensor(s) **560** may include different configurations, such as long range with narrow field of view, short range with wide field of view, short range side coverage, etc. In some examples, long-range RADAR may be used for adaptive cruise control functionality. The long-range RADAR systems may provide a broad field of view realized by two or more independent scans, such as within a 250 m range. The RADAR sensor(s) **560** may help in distinguishing between static and moving objects, and may be used by ADAS systems for emergency brake assist and forward collision warning. Long-range RADAR sensors may include monostatic multimodal RADAR with multiple (e.g., six or more) fixed RADAR antennae and a high-speed CAN and FlexRay interface. In an example with six antennae, the central four antennae may create a focused beam pattern, designed to record the vehicle's **500** surroundings at higher speeds with minimal interference from traffic in adjacent lanes. The other two antennae may expand the field of view,

making it possible to quickly detect vehicles entering or leaving the vehicle's **500** lane.

[0134] Mid-range RADAR systems may include, as an example, a range of up to 560 m (front) or 80 m (rear), and a field of view of up to 42 degrees (front) or 550 degrees (rear). Short-range RADAR systems may include, without limitation, RADAR sensors designed to be installed at both ends of the rear bumper. When installed at both ends of the rear bumper, such a RADAR sensor systems may create two beams that constantly monitor the blind spot in the rear and next to the vehicle.

[0135] Short-range RADAR systems may be used in an ADAS system for blind spot detection and/or lane change assist.

[0136] The vehicle **500** may further include ultrasonic sensor(s) **562**. The ultrasonic sensor(s) **562**, which may be positioned at the front, back, and/or the sides of the vehicle **500**, may be used for park assist and/or to create and update an occupancy grid. A wide variety of ultrasonic sensor(s) **562** may be used, and different ultrasonic sensor(s) **562** may be used for different ranges of detection (e.g., 2.5 m, 4 m). The ultrasonic sensor(s) **562** may operate at functional safety levels of ASIL B.

[0137] The vehicle **500** may include LIDAR sensor(s) **564**. The LIDAR sensor(s) **564** may be used for object and pedestrian detection, emergency braking, collision avoidance, and/or other functions. The LIDAR sensor(s) **564** may be functional safety level ASIL B. In some examples, the vehicle **500** may include multiple LIDAR sensors **564** (e.g., two, four, six, etc.) that may use Ethernet (e.g., to provide data to a Gigabit Ethernet switch).

[0138] In some examples, the LIDAR sensor(s) **564** may be capable of providing a list of objects and their distances for a 360-degree field of view. Commercially available LIDAR sensor(s) **564** may have an advertised range of approximately 500 m, with an accuracy of 2 cm-3 cm, and with support for a 500 Mbps Ethernet connection, for example. In some examples, one or more non-protruding LIDAR sensors **564** may be used. In such examples, the LIDAR sensor(s) **564** may be implemented as a small device that may be embedded into the front, rear, sides, and/or corners of the vehicle **500**. The LIDAR sensor(s) **564**, in such examples, may provide up to a 120-degree horizontal and 35-degree vertical field-of-view, with a 200 m range even for low-reflectivity objects. Front-mounted LIDAR sensor(s) **564** may be configured for a horizontal field of view between 45 degrees and 135 degrees.

[0139] In some examples, LIDAR technologies, such as 3D flash LIDAR, may also be used. 3D Flash LIDAR uses a flash of a laser as a transmission source, to illuminate vehicle surroundings up to approximately 200 m. A flash LIDAR unit includes a receptor, which records the laser pulse transit time and the reflected light on each pixel, which in turn corresponds to the range from the vehicle to the objects. Flash LIDAR may allow for highly accurate and distortion-free images of the surroundings to be generated with every laser flash. In some examples, four flash LIDAR sensors may be deployed, one at each side of the vehicle **500**. Available 3D flash LIDAR systems include a solid-state 3D staring array LIDAR camera with no moving parts other than a fan (e.g., a non-scanning LIDAR device). The flash LIDAR device may use a 5 nanosecond class I (eye-safe) laser pulse per frame and may capture the reflected laser light in the form of 3D range point clouds and co-registered intensity data. By using flash LIDAR, and because flash LIDAR is a solid-state device with no moving parts, the LIDAR sensor(s) **564** may be less susceptible to motion blur, vibration, and/or shock.

[0140] The vehicle may further include IMU sensor(s) **566**. The IMU sensor(s) **566** may be located at a center of the rear axle of the vehicle **500**, in some examples. The IMU sensor(s) **566** may include, for example and without limitation, an accelerometer(s), a magnetometer(s), a gyroscope(s), a magnetic compass(es), and/or other sensor types. In some examples, such as in six-axis applications, the IMU sensor(s) **566** may include accelerometers and gyroscopes, while in nine-axis applications, the IMU sensor(s) **566** may include accelerometers, gyroscopes, and magnetometers.

[0141] In some embodiments, the IMU sensor(s) **566** may be implemented as a miniature, high performance GPS-Aided Inertial Navigation System (GPS/INS) that combines micro-electro-mechanical systems (MEMS) inertial sensors, a high-sensitivity GPS receiver, and advanced Kalman filtering algorithms to provide estimates of position, velocity, and attitude. As such, in some examples, the IMU sensor(s) **566** may enable the vehicle **500** to estimate heading without requiring input from a magnetic sensor by directly observing and correlating the changes in velocity from GPS to the IMU sensor(s) **566**. In some examples, the IMU sensor(s) **566** and the GNSS sensor(s) **558** may be combined in a single integrated unit.

[0142] The vehicle may include microphone(s) **596** placed in and/or around the vehicle **500**. The microphone(s) **596** may be used for emergency vehicle detection and identification, among other things.

[0143] The vehicle may further include any number of camera types, including stereo camera(s) **568**, wide-view camera(s) **570**, infrared camera(s) **572**, surround camera(s) **574**, long-range and/or mid-range camera(s) **598**, and/or other camera types. The cameras may be used to capture image data around an entire periphery of the vehicle **500**. The types of cameras used depends on the embodiments and requirements for the vehicle **500**, and any combination of camera types may be used to provide the necessary coverage around the vehicle **500**. In addition, the number of cameras may differ depending on the embodiment. For example, the vehicle may include six cameras, seven cameras, ten cameras, twelve cameras, and/or another number of cameras. The cameras may support, as an example and without limitation, Gigabit Multimedia Serial Link (GMSL) and/or Gigabit Ethernet. Each of the camera(s) is described with more detail herein with respect to FIG. 5A and FIG. 5B.

[0144] The vehicle **500** may further include vibration sensor(s) **542**. The vibration sensor(s) **542** may measure vibrations of components of the vehicle, such as the axle(s). For example, changes in vibrations may indicate a change in road surfaces. In another example, when two or more vibration sensors **542** are used, the differences between the vibrations may be used to determine friction or slippage of the road surface (e.g., when the difference in vibration is between a power-driven axle and a freely rotating axle).

[0145] The vehicle **500** may include an ADAS system **538**. The ADAS system **538** may include a SoC, in some examples. The ADAS system **538** may include autonomous/adaptive/automatic cruise control (ACC), cooperative adaptive cruise control (CACC), forward crash warning (FCW), automatic emergency braking (AEB), lane departure warnings (LDW), lane keep assist (LKA), blind spot warning (BSW), rear cross-traffic warning (RCTW), collision warning systems (CWS), lane centering (LC), and/or other features and functionality.

[0146] The ACC systems may use RADAR sensor(s) **560**, LIDAR sensor(s) **564**, and/or a camera(s). The ACC systems may include longitudinal ACC and/or lateral ACC. Longitudinal ACC monitors and controls the distance to the vehicle immediately ahead of the vehicle **500** and automatically adjust the vehicle speed to maintain a safe distance from vehicles ahead. Lateral ACC performs distance keeping, and advises the vehicle **500** to change lanes when necessary. Lateral ACC is related to other ADAS applications such as LCA and CWS.

[0147] CACC uses information from other vehicles that may be received via the network interface **524** and/or the wireless antenna(s) **526** from other vehicles via a wireless link, or indirectly, over a network connection (e.g., over the Internet). Direct links may be provided by a vehicle-to-vehicle (V2V) communication link, while indirect links may be infrastructure-to-vehicle (I2V) communication link. In general, the V2V communication concept provides information about the immediately preceding vehicles (e.g., vehicles immediately ahead of and in the same lane as the vehicle **500**), while the I2V communication concept provides information about traffic further ahead. CACC systems may include either or both I2V and V2V information sources. Given the information of the vehicles ahead of the vehicle **500**, CACC may be more reliable and it has potential to improve traffic flow smoothness and reduce congestion on the road.

[0148] FCW systems are designed to alert the driver to a hazard, so that the driver may take corrective action. FCW systems use a front-facing camera and/or RADAR sensor(s) **560**, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component. FCW systems may provide a warning, such as in the form of a sound, visual warning, vibration and/or a quick brake pulse.

[0149] AEB systems detect an impending forward collision with another vehicle or other object, and may automatically apply the brakes if the driver does not take corrective action within a specified time or distance parameter. AEB systems may use front-facing camera(s) and/or RADAR sensor(s) **560**, coupled to a dedicated processor, DSP, FPGA, and/or ASIC. When the AEB system detects a hazard, it typically first alerts the driver to take corrective action to avoid the collision and, if the driver does not take corrective action, the AEB system may automatically apply the brakes in an effort to prevent, or at least mitigate, the impact of the predicted collision. AEB systems, may include techniques such as dynamic brake support and/or crash imminent braking.

[0150] LDW systems provide visual, audible, and/or tactile warnings, such as steering wheel or seat vibrations, to alert the driver when the vehicle **500** crosses lane markings. A LDW system does not activate when the driver indicates an intentional lane departure, by activating a turn signal. LDW systems may use front-side facing cameras, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0151] LKA systems are a variation of LDW systems. LKA systems provide steering input or braking to correct the vehicle **500** if the vehicle **500** starts to exit the lane.

[0152] BSW systems detects and warn the driver of vehicles in an automobile's blind spot. BSW systems may provide a visual, audible, and/or tactile alert to indicate that merging or changing lanes is unsafe. The system may provide an additional warning when the driver uses a turn signal. BSW systems may use rear-side facing camera(s) and/or RADAR sensor(s) **560**, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0153] RCTW systems may provide visual, audible, and/or tactile notification when an object is detected outside the rear-camera range when the vehicle **500** is backing up. Some RCTW systems include AEB to ensure that the vehicle brakes are applied to avoid a crash. RCTW systems may use one or more rear-facing RADAR sensor(s) **560**, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0154] Conventional ADAS systems may be prone to false positive results which may be annoying and distracting to a driver, but typically are not catastrophic, because the ADAS systems alert the driver and allow the driver to decide whether a safety condition truly exists and act accordingly. However, in an autonomous vehicle **500**, the vehicle **500** itself must, in the case of conflicting results, decide whether to heed the result from a primary computer or a secondary computer (e.g., a first controller **536** or a second controller **536**). For example, in some embodiments, the ADAS system **538** may be a backup and/or secondary computer for providing perception information to a backup computer rationality module. The backup computer rationality monitor may run a redundant diverse software on hardware components to detect faults in perception and dynamic driving tasks. Outputs from the ADAS system **538** may be provided to a supervisory MCU. If outputs from the primary computer and the secondary computer conflict, the supervisory MCU must determine how to reconcile the conflict to ensure safe operation.

[0155] In some examples, the primary computer may be configured to provide the supervisory MCU with a confidence score, indicating the primary computer's confidence in the chosen result. If the confidence score exceeds a threshold, the supervisory MCU may follow the primary computer's direction, regardless of whether the secondary computer provides a conflicting or inconsistent result. Where the confidence score does not meet the threshold, and where the primary and

secondary computer indicate different results (e.g., the conflict), the supervisory MCU may arbitrate between the computers to determine the appropriate outcome.

[0156] The supervisory MCU may be configured to run a neural network(s) that is trained and configured to determine, based on outputs from the primary computer and the secondary computer, conditions under which the secondary computer provides false alarms. Thus, the neural network(s) in the supervisory MCU may learn when the secondary computer's output may be trusted, and when it cannot. For example, when the secondary computer is a RADAR-based FCW system, a neural network(s) in the supervisory MCU may learn when the FCW system is identifying metallic objects that are not, in fact, hazards, such as a drainage grate or manhole cover that triggers an alarm. Similarly, when the secondary computer is a camera-based LDW system, a neural network in the supervisory MCU may learn to override the LDW when bicyclists or pedestrians are present and a lane departure is, in fact, the safest maneuver. In embodiments that include a neural network(s) running on the supervisory MCU, the supervisory MCU may include at least one of a DLA or GPU suitable for running the neural network(s) with associated memory. In preferred embodiments, the supervisory MCU may comprise and/or be included as a component of the SoC(s) **504**.

[0157] In other examples, ADAS system **538** may include a secondary computer that performs ADAS functionality using traditional rules of computer vision. As such, the secondary computer may use classic computer vision rules (if-then), and the presence of a neural network(s) in the supervisory MCU may improve reliability, safety and performance. For example, the diverse implementation and intentional non-identity makes the overall system more fault-tolerant, especially to faults caused by software (or software-hardware interface) functionality. For example, if there is a software bug or error in the software running on the primary computer, and the non-identical software code running on the secondary computer provides the same overall result, the supervisory MCU may have greater confidence that the overall result is correct, and the bug in software or hardware on primary computer is not causing material error.

[0158] In some examples, the output of the ADAS system **538** may be fed into the primary computer's perception block and/or the primary computer's dynamic driving task block. For example, if the ADAS system **538** indicates a forward crash warning due to an object immediately ahead, the perception block may use this information when identifying objects. In other examples, the secondary computer may have its own neural network which is trained and thus reduces the risk of false positives, as described herein.

[0159] The vehicle **500** may further include the infotainment SoC **530** (e.g., an in-vehicle infotainment system (IVI)). Although illustrated and described as a SoC, the infotainment system may not be a SoC, and may include two or more discrete components. The infotainment SoC **530** may include a combination of hardware and software that may be used to provide audio (e.g., music, a personal digital assistant, navigational instructions, news, radio, etc.), video (e.g., TV, movies, streaming, etc.), phone (e.g., hands-free calling), network connectivity (e.g., LTE, Wi-Fi, etc.), and/or information services (e.g., navigation systems, rear-parking assistance, a radio data system, vehicle related information such as fuel level, total distance covered, brake fuel level, oil level, door open/close, air filter information, etc.) to the vehicle **500**. For example, the infotainment SoC **530** may radios, disk players, navigation systems, video players, USB and Bluetooth connectivity, carputers, in-car entertainment, Wi-Fi, steering wheel audio controls, hands free voice control, a heads-up display (HUD), an HMI display **534**, a telematics device, a control panel (e.g., for controlling and/or interacting with various components, features, and/or systems), and/or other components. The infotainment SoC **530** may further be used to provide information (e.g., visual and/or audible) to a user(s) of the vehicle, such as information from the ADAS system **538**, autonomous driving information such as planned vehicle maneuvers, trajectories, surrounding environment information (e.g., intersection information, vehicle information, road information, etc.), and/or other information.

[0160] The infotainment SoC **530** may include GPU functionality. The infotainment SoC **530** may communicate over the bus **502** (e.g., CAN bus, Ethernet, etc.) with other devices, systems, and/or components of the vehicle **500**. In some examples, the infotainment SoC **530** may be coupled to a supervisory MCU such that the GPU of the infotainment system may perform some self-driving functions in the event that the primary controller(s) **536** (e.g., the primary and/or backup computers of the vehicle **500**) fail. In such an example, the infotainment SoC **530** may put the vehicle **500** into a chauffeur to safe stop mode, as described herein.

[0161] The vehicle **500** may further include an instrument cluster **532** (e.g., a digital dash, an electronic instrument cluster, a digital instrument panel, etc.). The instrument cluster **532** may include a controller and/or supercomputer (e.g., a discrete controller or supercomputer). The instrument cluster **532** may include a set of instrumentation such as a speedometer, fuel level, oil pressure, tachometer, odometer, turn indicators, gearshift position indicator, seat belt warning light(s), parking-brake warning light(s), engine-malfunction light(s), airbag (SRS) system information, lighting controls, safety system controls, navigation information, etc. In some examples, information may be displayed and/or shared among the infotainment SoC **530** and the instrument cluster **532**. In other words, the instrument cluster **532** may be included as part of the infotainment SoC **530**, or vice versa.

[0162] FIG. 5D is a system diagram for communication between cloud-based server(s) and the example autonomous vehicle **500** of FIG. 5A, in accordance with some embodiments of the present disclosure. The system **576** may include server(s) **578**, network(s) **590**, and vehicles, including the vehicle **500**. The server(s) **578** may include a plurality of GPUs **584(A)-584(H)** (collectively referred to herein as GPUs **584**), PCIe switches **582(A)-582(H)** (collectively referred to herein as PCIe switches **582**), and/or CPUs **580(A)-580(B)** (collectively referred to herein as CPUs **580**). The GPUs **584**, the CPUs **580**, and the PCIe switches may be interconnected with high-speed interconnects such as, for example and without limitation, NVLink interfaces **588** developed by NVIDIA and/or PCIe connections **586**. In some examples, the GPUs **584** are connected via NVLink and/or NVSwitch SoC and the GPUs **584** and the PCIe switches **582** are connected via PCIe interconnects. Although eight GPUs **584**, two CPUs **580**, and two PCIe switches are illustrated, this is not intended to be limiting. Depending on the embodiment, each of the server(s) **578** may include any number of GPUs **584**, CPUs **580**, and/or PCIe switches. For example, the server(s) **578** may each include eight, sixteen, thirty-two, and/or more GPUs **584**.

[0163] The server(s) **578** may receive, over the network(s) **590** and from the vehicles, image data representative of images showing unexpected or changed road conditions, such as recently commenced road-work. The server(s) **578** may transmit, over the network(s) **590** and to the vehicles, neural networks **592**, updated neural networks **592**, and/or map information **594**, including information regarding traffic and road conditions. The updates to the map information **594** may include updates for the HD map **522**, such as information regarding construction sites, potholes, detours, flooding, and/or other obstructions. In some examples, the neural networks **592**, the updated neural networks **592**, and/or the map information **594** may have resulted from new training and/or experiences represented in data received from any number of vehicles in the environment, and/or based on training performed at a datacenter (e.g., using the server(s) **578** and/or other servers).

[0164] The server(s) **578** may be used to train MLM(s) (e.g., neural networks) based on training data. The training data may be generated by the vehicles, and/or may be generated in a simulation (e.g., using a game engine). In some examples, the training data is tagged (e.g., where the neural network benefits from supervised learning) and/or undergoes other pre-processing, while in other examples the training data is not tagged and/or pre-processed (e.g., where the neural network does not require supervised learning). Training may be executed according to any one or more classes of machine learning techniques, including, without limitation, classes such as: supervised training, semi-supervised training, unsupervised training, self-learning, reinforcement learning, federated

learning, transfer learning, feature learning (including principal component and cluster analyses), multi-linear subspace learning, manifold learning, representation learning (including sparse dictionary learning), rule-based machine learning, anomaly detection, and any variants or combinations thereof. Once the machine learning models are trained, the machine learning models may be used by the vehicles (e.g., transmitted to the vehicles over the network(s) **590**, and/or the machine learning models may be used by the server(s) **578** to remotely monitor the vehicles.

[0165] In some examples, the server(s) **578** may receive data from the vehicles and apply the data to up-to-date real-time neural networks for real-time intelligent inferencing. The server(s) **578** may include deep-learning supercomputers and/or dedicated AI computers powered by GPU(s) **584**, such as a DGX and DGX Station machines developed by NVIDIA. However, in some examples, the server(s) **578** may include deep learning infrastructure that use only CPU-powered datacenters.

[0166] The deep-learning infrastructure of the server(s) **578** may be capable of fast, real-time inferencing, and may use that capability to evaluate and verify the health of the processors, software, and/or associated hardware in the vehicle **500**. For example, the deep-learning infrastructure may receive periodic updates from the vehicle **500**, such as a sequence of images and/or objects that the vehicle **500** has located in that sequence of images (e.g., via computer vision and/or other machine learning object classification techniques). The deep-learning infrastructure may run its own neural network to identify the objects and compare them with the objects identified by the vehicle **500** and, if the results do not match and the infrastructure concludes that the AI in the vehicle **500** is malfunctioning, the server(s) **578** may transmit a signal to the vehicle **500** instructing a fail-safe computer of the vehicle **500** to assume control, notify the passengers, and complete a safe parking maneuver.

[0167] For inferencing, the server(s) **578** may include the GPU(s) **584** and one or more programmable inference accelerators (e.g., NVIDIA's TensorRT). The combination of GPU-powered servers and inference acceleration may make real-time responsiveness possible. In other examples, such as where performance is less critical, servers powered by CPUs, FPGAs, and other processors may be used for inferencing.

Example Computing Device

[0168] FIG. **6** is a block diagram of an example computing device(s) **600** suitable for use in implementing some embodiments of the present disclosure. Computing device **600** may include an interconnect system **602** that directly or indirectly couples the following devices: memory **604**, one or more central processing units (CPUs) **606**, one or more graphics processing units (GPUs) **608**, a communication interface **610**, input/output (I/O) ports **612**, input/output components **614**, a power supply **616**, one or more presentation components **618** (e.g., display(s)), and one or more logic units **620**. In at least one embodiment, the computing device(s) **600** may comprise one or more virtual machines (VMs), and/or any of the components thereof may comprise virtual components (e.g., virtual hardware components). For non-limiting examples, one or more of the GPUs **608** may comprise one or more vGPUs, one or more of the CPUs **606** may comprise one or more vCPUs, and/or one or more of the logic units **620** may comprise one or more virtual logic units. As such, a computing device(s) **600** may include discrete components (e.g., a full GPU dedicated to the computing device **600**), virtual components (e.g., a portion of a GPU dedicated to the computing device **600**), or a combination thereof.

[0169] Although the various blocks of FIG. **6** are shown as connected via the interconnect system **602** with lines, this is not intended to be limiting and is for clarity only. For example, in some embodiments, a presentation component **618**, such as a display device, may be considered an I/O component **614** (e.g., if the display is a touch screen). As another example, the CPUs **606** and/or GPUs **608** may include memory (e.g., the memory **604** may be representative of a storage device in addition to the memory of the GPUs **608**, the CPUs **606**, and/or other components). In other words, the computing device of FIG. **6** is merely illustrative. Distinction is not made between such categories as “workstation,” “server,” “laptop,” “desktop,” “tablet,” “client device,” “mobile

device,” “hand-held device,” “game console,” “electronic control unit (ECU),” “virtual reality system,” and/or other device or system types, as all are contemplated within the scope of the computing device of FIG. 6.

[0170] The interconnect system **602** may represent one or more links or busses, such as an address bus, a data bus, a control bus, or a combination thereof. The interconnect system **602** may include one or more bus or link types, such as an industry standard architecture (ISA) bus, an extended industry standard architecture (EISA) bus, a video electronics standards association (VESA) bus, a peripheral component interconnect (PCI) bus, a peripheral component interconnect express (PCIe) bus, and/or another type of bus or link. In some embodiments, there are direct connections between components. As an example, the CPU **606** may be directly connected to the memory **604**. Further, the CPU **606** may be directly connected to the GPU **608**. Where there is direct, or point-to-point connection between components, the interconnect system **602** may include a PCIe link to carry out the connection. In these examples, a PCI bus need not be included in the computing device **600**.

[0171] The memory **604** may include any of a variety of computer-readable media. The computer-readable media may be any available media that may be accessed by the computing device **600**. The computer-readable media may include both volatile and nonvolatile media, and removable and non-removable media. By way of example, and not limitation, the computer-readable media may comprise computer-storage media and communication media.

[0172] The computer-storage media may include both volatile and nonvolatile media and/or removable and non-removable media implemented in any method or technology for storage of information such as computer-readable instructions, data structures, program modules, and/or other data types. For example, the memory **604** may store computer-readable instructions (e.g., that represent a program(s) and/or a program element(s), such as an operating system. Computer-storage media may include, but is not limited to, RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium which may be used to store the desired information and which may be accessed by computing device **600**. As used herein, computer storage media does not comprise signals per se.

[0173] The computer storage media may embody computer-readable instructions, data structures, program modules, and/or other data types in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. The term “modulated data signal” may refer to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal. By way of example, and not limitation, the computer storage media may include wired media such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media. Combinations of any of the above should also be included within the scope of computer-readable media.

[0174] The CPU(s) **606** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **600** to perform one or more of the methods and/or processes described herein. The CPU(s) **606** may each include one or more cores (e.g., one, two, four, eight, twenty-eight, seventy-two, etc.) that are capable of handling a multitude of software threads simultaneously. The CPU(s) **606** may include any type of processor, and may include different types of processors depending on the type of computing device **600** implemented (e.g., processors with fewer cores for mobile devices and processors with more cores for servers). For example, depending on the type of computing device **600**, the processor may be an Advanced RISC Machines (ARM) processor implemented using Reduced Instruction Set Computing (RISC) or an x86 processor implemented using Complex Instruction Set Computing (CISC). The computing device **600** may include one or more CPUs **606** in addition to one or more microprocessors or supplementary co-processors, such as math co-processors.

[0175] In addition to or alternatively from the CPU(s) **606**, the GPU(s) **608** may be configured to

execute at least some of the computer-readable instructions to control one or more components of the computing device **600** to perform one or more of the methods and/or processes described herein. One or more of the GPU(s) **608** may be an integrated GPU (e.g., with one or more of the CPU(s) **606** and/or one or more of the GPU(s) **608** may be a discrete GPU. In embodiments, one or more of the GPU(s) **608** may be a coprocessor of one or more of the CPU(s) **606**. The GPU(s) **608** may be used by the computing device **600** to render graphics (e.g., 3D graphics) or perform general purpose computations. For example, the GPU(s) **608** may be used for General-Purpose computing on GPUs (GPGPU). The GPU(s) **608** may include hundreds or thousands of cores that are capable of handling hundreds or thousands of software threads simultaneously. The GPU(s) **608** may generate pixel data for output images in response to rendering commands (e.g., rendering commands from the CPU(s) **606** received via a host interface). The GPU(s) **608** may include graphics memory, such as display memory, for storing pixel data or any other suitable data, such as GPGPU data. The display memory may be included as part of the memory **604**. The GPU(s) **608** may include two or more GPUs operating in parallel (e.g., via a link). The link may directly connect the GPUs (e.g., using NVLINK) or may connect the GPUs through a switch (e.g., using NVSwitch). When combined together, each GPU **608** may generate pixel data or GPGPU data for different portions of an output or for different outputs (e.g., a first GPU for a first image and a second GPU for a second image). Each GPU may include its own memory, or may share memory with other GPUs.

[0176] In addition to or alternatively from the CPU(s) **606** and/or the GPU(s) **608**, the logic unit(s) **620** may be configured to execute at least some of the computer-readable instructions to control one or more components of the computing device **600** to perform one or more of the methods and/or processes described herein. In embodiments, the CPU(s) **606**, the GPU(s) **608**, and/or the logic unit(s) **620** may discretely or jointly perform any combination of the methods, processes and/or portions thereof. One or more of the logic units **620** may be part of and/or integrated in one or more of the CPU(s) **606** and/or the GPU(s) **608** and/or one or more of the logic units **620** may be discrete components or otherwise external to the CPU(s) **606** and/or the GPU(s) **608**. In embodiments, one or more of the logic units **620** may be a coprocessor of one or more of the CPU(s) **606** and/or one or more of the GPU(s) **608**.

[0177] Examples of the logic unit(s) **620** include one or more processing cores and/or components thereof, such as Data Processing Units (DPUs), Tensor Cores (TCs), Tensor Processing Units (TPUs), Pixel Visual Cores (PVCs), Vision Processing Units (VPUs), Graphics Processing Clusters (GPCs), Texture Processing Clusters (TPCs), Streaming Multiprocessors (SMs), Tree Traversal Units (TTUs), Artificial Intelligence Accelerators (AIAs), Deep Learning Accelerators (DLAs), Arithmetic-Logic Units (ALUs), Application-Specific Integrated Circuits (ASICs), Floating Point Units (FPUs), input/output (I/O) elements, peripheral component interconnect (PCI) or peripheral component interconnect express (PCIe) elements, and/or the like.

[0178] The communication interface **610** may include one or more receivers, transmitters, and/or transceivers that enable the computing device **600** to communicate with other computing devices via an electronic communication network, included wired and/or wireless communications. The communication interface **610** may include components and functionality to enable communication over any of a number of different networks, such as wireless networks (e.g., Wi-Fi, Z-Wave, Bluetooth, Bluetooth LE, ZigBee, etc.), wired networks (e.g., communicating over Ethernet or InfiniBand), low-power wide-area networks (e.g., LoRaWAN, SigFox, etc.), and/or the Internet. In one or more embodiments, logic unit(s) **620** and/or communication interface **610** may include one or more data processing units (DPUs) to transmit data received over a network and/or through interconnect system **602** directly to (e.g., a memory of) one or more GPU(s) **608**.

[0179] The I/O ports **612** may enable the computing device **600** to be logically coupled to other devices including the I/O components **614**, the presentation component(s) **618**, and/or other components, some of which may be built in to (e.g., integrated in) the computing device **600**.

Illustrative I/O components **614** include a microphone, mouse, keyboard, joystick, game pad, game controller, satellite dish, scanner, printer, wireless device, etc. The I/O components **614** may provide a natural user interface (NUI) that processes air gestures, voice, or other physiological inputs generated by a user. In some instances, inputs may be transmitted to an appropriate network element for further processing. An NUI may implement any combination of speech recognition, stylus recognition, facial recognition, biometric recognition, gesture recognition both on screen and adjacent to the screen, air gestures, head and eye tracking, and touch recognition (as described in more detail below) associated with a display of the computing device **600**. The computing device **600** may include depth cameras, such as stereoscopic camera systems, infrared camera systems, RGB camera systems, touchscreen technology, and combinations of these, for gesture detection and recognition. Additionally, the computing device **600** may include accelerometers or gyroscopes (e.g., as part of an inertia measurement unit (IMU)) that enable detection of motion. In some examples, the output of the accelerometers or gyroscopes may be used by the computing device **600** to render immersive augmented reality or virtual reality.

[0180] The power supply **616** may include a hard-wired power supply, a battery power supply, or a combination thereof. The power supply **616** may provide power to the computing device **600** to enable the components of the computing device **600** to operate.

[0181] The presentation component(s) **618** may include a display (e.g., a monitor, a touch screen, a television screen, a heads-up-display (HUD), other display types, or a combination thereof), speakers, and/or other presentation components. The presentation component(s) **618** may receive data from other components (e.g., the GPU(s) **608**, the CPU(s) **606**, DPUs, etc.), and output the data (e.g., as an image, video, sound, etc.).

Example Data Center

[0182] FIG. 7 illustrates an example data center **700** that may be used in at least one embodiment of the present disclosure. The data center **700** may include a data center infrastructure layer **710**, a framework layer **720**, a software layer **730**, and/or an application layer **740**.

[0183] As shown in FIG. 7, the data center infrastructure layer **710** may include a resource orchestrator **712**, grouped computing resources **714**, and node computing resources (“node C.R.s”) **716(1)-716(N)**, where “N” represents any whole, positive integer. In at least one embodiment, node C.R.s **716(1)-716(N)** may include, but are not limited to, any number of central processing units (CPUs) or other processors (including DPUs, accelerators, field programmable gate arrays (FPGAs), graphics processors or graphics processing units (GPUs), etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output (NW I/O) devices, network switches, virtual machines (VMs), power modules, and/or cooling modules, etc. In some embodiments, one or more node C.R.s from among node C.R.s **716(1)-716(N)** may correspond to a server having one or more of the above-mentioned computing resources. In addition, in some embodiments, the node C.R.s **716(1)-716(N)** may include one or more virtual components, such as vGPUs, vCPUs, and/or the like, and/or one or more of the node C.R.s **716(1)-716(N)** may correspond to a virtual machine (VM).

[0184] In at least one embodiment, grouped computing resources **714** may include separate groupings of node C.R.s **716** housed within one or more racks (not shown), or many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s **716** within grouped computing resources **714** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s **716** including CPUs, GPUs, DPUs, and/or other processors may be grouped within one or more racks to provide compute resources to support one or more workloads. The one or more racks may also include any number of power modules, cooling modules, and/or network switches, in any combination.

[0185] The resource orchestrator **712** may configure or otherwise control one or more node C.R.s **716(1)-716(N)** and/or grouped computing resources **714**. In at least one embodiment, resource

orchestrator **712** may include a software design infrastructure (SDI) management entity for the data center **700**. The resource orchestrator **712** may include hardware, software, or some combination thereof.

[0186] In at least one embodiment, as shown in FIG. 7, framework layer **720** may include a job scheduler **733**, a configuration manager **734**, a resource manager **736**, and/or a distributed file system **738**. The framework layer **720** may include a framework to support software **732** of software layer **730** and/or one or more application(s) **742** of application layer **740**. The software **732** or application(s) **742** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. The framework layer **720** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter “Spark”) that may utilize distributed file system **738** for large-scale data processing (e.g., “big data”). In at least one embodiment, job scheduler **733** may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center **700**. The configuration manager **734** may be capable of configuring different layers such as software layer **730** and framework layer **720** including Spark and distributed file system **738** for supporting large-scale data processing. The resource manager **736** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **738** and job scheduler **733**. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource **714** at data center infrastructure layer **710**. The resource manager **736** may coordinate with resource orchestrator **712** to manage these mapped or allocated computing resources.

[0187] In at least one embodiment, software **732** included in software layer **730** may include software used by at least portions of node C.R.s **716(1)-716(N)**, grouped computing resources **714**, and/or distributed file system **738** of framework layer **720**. One or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0188] In at least one embodiment, application(s) **742** included in application layer **740** may include one or more types of applications used by at least portions of node C.R.s **716(1)-716(N)**, grouped computing resources **714**, and/or distributed file system **738** of framework layer **720**. One or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.), and/or other machine learning applications used in conjunction with one or more embodiments.

[0189] In at least one embodiment, any of configuration manager **734**, resource manager **736**, and resource orchestrator **712** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. Self-modifying actions may relieve a data center operator of data center **700** from making possibly bad configuration decisions and possibly avoiding underutilized and/or poor performing portions of a data center.

[0190] The data center **700** may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein. For example, a machine learning model(s) may be trained by calculating weight parameters according to a neural network architecture using software and/or computing resources described above with respect to the data center **700**. In at least one embodiment, trained or deployed machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to the data center **700** by using weight parameters calculated through one or more training techniques, such as but not limited to those described herein.

[0191] In at least one embodiment, the data center **700** may use CPUs, application-specific

integrated circuits (ASICs), GPUs, FPGAs, and/or other hardware (or virtual compute resources corresponding thereto) to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

Example Network Environments

[0192] Network environments suitable for use in implementing embodiments of the disclosure may include one or more client devices, servers, network attached storage (NAS), other backend devices, and/or other device types. The client devices, servers, and/or other device types (e.g., each device) may be implemented on one or more instances of the computing device(s) **600** of FIG. 6—e.g., each device may include similar components, features, and/or functionality of the computing device(s) **600**. In addition, where backend devices (e.g., servers, NAS, etc.) are implemented, the backend devices may be included as part of a data center **700**, an example of which is described in more detail herein with respect to FIG. 7.

[0193] Components of a network environment may communicate with each other via a network(s), which may be wired, wireless, or both. The network may include multiple networks, or a network of networks. By way of example, the network may include one or more Wide Area Networks (WANs), one or more Local Area Networks (LANs), one or more public networks such as the Internet and/or a public switched telephone network (PSTN), and/or one or more private networks. Where the network includes a wireless telecommunications network, components such as a base station, a communications tower, or even access points (as well as other components) may provide wireless connectivity.

[0194] Compatible network environments may include one or more peer-to-peer network environments—in which case a server may not be included in a network environment—and one or more client-server network environments—in which case one or more servers may be included in a network environment. In peer-to-peer network environments, functionality described herein with respect to a server(s) may be implemented on any number of client devices.

[0195] In at least one embodiment, a network environment may include one or more cloud-based network environments, a distributed computing environment, a combination thereof, etc. A cloud-based network environment may include a framework layer, a job scheduler, a resource manager, and a distributed file system implemented on one or more of servers, which may include one or more core network servers and/or edge servers. A framework layer may include a framework to support software of a software layer and/or one or more application(s) of an application layer. The software or application(s) may respectively include web-based service software or applications. In embodiments, one or more of the client devices may use the web-based service software or applications (e.g., by accessing the service software and/or applications via one or more application programming interfaces (APIs)). The framework layer may be, but is not limited to, a type of free and open-source software web application framework such as that may use a distributed file system for large-scale data processing (e.g., “big data”).

[0196] A cloud-based network environment may provide cloud computing and/or cloud storage that carries out any combination of computing and/or data storage functions described herein (or one or more portions thereof). Any of these various functions may be distributed over multiple locations from central or core servers (e.g., of one or more data centers that may be distributed across a state, a region, a country, the globe, etc.). If a connection to a user (e.g., a client device) is relatively close to an edge server(s), a core server(s) may designate at least a portion of the functionality to the edge server(s). A cloud-based network environment may be private (e.g., limited to a single organization), may be public (e.g., available to many organizations), and/or a combination thereof (e.g., a hybrid cloud environment).

[0197] The client device(s) may include at least some of the components, features, and functionality of the example computing device(s) **600** described herein with respect to FIG. 6. By

way of example and not limitation, a client device may be embodied as a Personal Computer (PC), a laptop computer, a mobile device, a smartphone, a tablet computer, a smart watch, a wearable computer, a Personal Digital Assistant (PDA), an MP3 player, a virtual reality headset, a Global Positioning System (GPS) or device, a video player, a video camera, a surveillance device or system, a vehicle, a boat, a flying vessel, a virtual machine, a drone, a robot, a handheld communications device, a hospital device, a gaming device or system, an entertainment system, a vehicle computer system, an embedded system controller, a remote control, an appliance, a consumer electronic device, a workstation, an edge device, any combination of these delineated devices, or any other suitable device.

[0198] The disclosure may be described in the general context of computer code or machine-useable instructions, including computer-executable instructions such as program modules, being executed by a computer or other machine, such as a personal data assistant or other handheld device. Generally, program modules including routines, programs, objects, components, data structures, etc., refer to code that perform particular tasks or implement particular abstract data types. The disclosure may be practiced in a variety of system configurations, including hand-held devices, consumer electronics, general-purpose computers, more specialty computing devices, etc. The disclosure may also be practiced in distributed computing environments where tasks are performed by remote-processing devices that are linked through a communications network.

[0199] As used herein, a recitation of “and/or” with respect to two or more elements should be interpreted to mean only one element, or a combination of elements. For example, “element A, element B, and/or element C” may include only element A, only element B, only element C, element A and element B, element A and element C, element B and element C, or elements A, B, and C. In addition, “at least one of element A or element B” may include at least one of element A, at least one of element B, or at least one of element A and at least one of element B. Further, “at least one of element A and element B” may include at least one of element A, at least one of element B, or at least one of element A and at least one of element B.

[0200] The subject matter of the present disclosure is described with specificity herein to meet statutory requirements. However, the description itself is not intended to limit the scope of this disclosure. Rather, the inventors have contemplated that the claimed subject matter might also be embodied in other ways, to include different steps or combinations of steps similar to the ones described in this document, in conjunction with other present or future technologies. Moreover, although the terms “step” and/or “block” may be used herein to connote different elements of methods employed, the terms should not be interpreted as implying any particular order among or between various steps herein disclosed unless and except when the order of individual steps is explicitly described.

Claims

1. A method comprising: computing, using one or more machine learning models (MLMs) and based at least on images obtained using one or more sensors of a machine, one or more likelihoods of correspondence between pixels of the images; computing, using the one or more likelihoods of correspondence, a relative sensor pose of the one or more sensors; determining, using the relative sensor pose, one or more depth values associated with the images; and performing one or more planning, navigation, or control operations corresponding to the machine based at least on the one or more depth values.
2. The method of claim 1, wherein the computing of the relative sensor pose includes: estimating an essential matrix using the one or more likelihoods of correspondence; and determining the relative sensor pose using the essential matrix.
3. The method of claim 1, wherein the determining of the one or more depth values includes: identifying one or more corresponding pixels between the images based at least on the one or more

likelihoods of correspondence; and computing the one or more depth values using the relative sensor pose and geometric relationships between the one or more corresponding pixels.

4. The method of claim 1, wherein the determining of the one or more depth values includes triangulating positions of corresponding pixels between the images using the relative sensor pose.

5. The method of claim 1, wherein the one or more likelihoods of correspondence include a plurality of likelihoods of correspondences for a plurality of matching pixel candidates between the images, and the computing of the relative sensor pose includes: selecting, using the plurality of likelihoods of correspondences, a subset of the plurality of matching pixel candidates as matching pixels between the images; and determining the relative sensor pose using the matching pixels between the images.

6. The method of claim 5, further comprising determining the plurality of matching pixel candidates using a second relative sensor pose, the second relative sensor pose estimated based at least on applying optical flow estimation to the images.

7. The method of claim 1, wherein the computing of one or more likelihoods of correspondence between pixels of the images is based at least on a second relative camera pose, and the method further includes: identifying one or more pixels of the pixels that correspond to image features that are invariant to at least one of scale or rotation transformations between the images; selecting, from potential correspondences between the images, a subset of correspondences that correspond to the identified one or more pixels; and determining the second relative camera pose using the selected subset of correspondences.

8. The method of claim 1, wherein the computing of one or more likelihoods of correspondence between the pixels of the images is based at least on a second relative camera pose, and the method further includes: analyzing feature responses across multiple scales or orientations of the images to detect, in the images, one or more keypoints corresponding to image features that are invariant to at least one of scale or rotation transformations between the images; identifying, from a plurality of potential correspondences between the images, a subset of correspondences that correspond to the detected one or more keypoints; and determining the second relative camera pose using the identified subset of correspondences.

9. The method of claim 1, wherein the one or more likelihoods of correspondence include a plurality of likelihoods of correspondence and the computing of the relative sensor pose includes: aggregating the plurality of likelihoods of correspondence for one or more regions around one or more pixels to determine one or more aggregated values for the one or more pixels, and determining the relative sensor pose using the one or more aggregated values.

10. The method of claim 1, wherein the one or more likelihoods of correspondence include a plurality of likelihoods of correspondence corresponding to different warpings of a first image of the images to locations in a second image of the images, each warping of the warpings corresponding to a respective hypothesized depth plane.

11. An autonomous or semi-autonomous machine comprising: one or more central processing units (CPUs); one or more graphics processing units (GPUs); one or more hardware accelerators; and one or more external sensors having one or more fields of view or one or more sensory fields external to the autonomous or semi-autonomous machine, wherein the autonomous or semi-autonomous machine is to: compute, based at least on one or more machine learning models (MLMs) processing sensor data obtained using the one or more external sensors, one or more likelihoods of correspondence between pixels of images; compute, using the one or more likelihoods of correspondence, a relative sensor pose of the one or more sensors; determine, using the relative sensor pose, one or more depth values associated with the images; and perform one or more planning, navigation, or control operations based at least on the one or more depth values.

12. The autonomous or semi-autonomous machine of claim 11, wherein the computing of the relative sensor pose includes: estimating an essential matrix using the one or more likelihoods of correspondence; and determining the relative sensor pose using the essential matrix.

- 13.** The autonomous or semi-autonomous machine of claim 11, wherein the determining of the one or more depth values includes: identifying one or more corresponding pixels between the images based at least on the one or more likelihoods of correspondence; and computing the one or more depth values using the relative sensor pose and geometric relationships between the one or more corresponding pixels.
- 14.** The autonomous or semi-autonomous machine of claim 11, wherein the determining of the one or more depth values includes triangulating positions of corresponding pixels between the images using the relative sensor pose.
- 15.** The autonomous or semi-autonomous machine of claim 11, wherein the one or more likelihoods of correspondence include a plurality of likelihoods of correspondences for a plurality of matching pixel candidates between the images, and the computing of the relative sensor pose includes: selecting, using the plurality of likelihoods of correspondences, a subset of the plurality of matching pixel candidates as matching pixels between the images; and determining the relative sensor pose using the matching pixels between the images.
- 16.** A system comprising: one or more central processing units (CPUs); one or more graphics processing units (GPUs); one or more hardware accelerators; and one or more external sensors having one or more fields of view or one or more sensory fields, wherein the system causes a machine to perform one or more operations based at least on one or more depth values associated with images obtained using the one or more external sensors, the one or more depth values determined using a relative sensor pose that corresponds to one or more likelihoods of correspondence between pixels of the images, and the one or more likelihoods of correspondence computed based at least on the images and using the one or more machine learning models (MLMs).
- 17.** The system of claim 16, wherein the relative sensor pose is computed based at least on: estimating an essential matrix using the one or more likelihoods of correspondence; and determining the relative sensor pose using the essential matrix.
- 18.** The system of claim 16, wherein the one or more depth values are determined based at least on: identifying one or more corresponding pixels between the images based at least on the one or more likelihoods of correspondence; and computing the one or more depth values using the relative sensor pose and geometric relationships between the one or more corresponding pixels.
- 19.** The system of claim 16, wherein the one or more depth values are determined based at least on triangulating positions of corresponding pixels between the images using the relative sensor pose.
- 20.** The system of claim 16, wherein the system is comprised in at least one of: a control system for an autonomous or semi-autonomous machine; a perception system for an autonomous or semi-autonomous machine; a system for performing one or more simulation operations; a system for performing light transport simulation; a system for performing collaborative content creation for 3D assets; a system for performing one or more deep learning operations; a system implemented using an edge device; a system implemented using a robot; a system for performing one or more generative AI operations; a system for presenting at least one of virtual reality content, augmented reality content, or mixed reality content; a system incorporating one or more virtual machines (VMs); a system implemented at least partially in a data center; or a system implemented at least partially using cloud computing resources.
-